
Azure Databricks ETL Pipeline

End-to-End ETL Pipeline Using Azure Databricks, Azure Data Factory and ADLS Gen2



Prepared By: Ashwini Chavan

Date: 14-July- 2026

Version: 1.0

Contents

1	Objective	4
2	Technologies Used	4
3	High Level Business Requirements	5
4	Solution Architecture	6
5	Project Architecture Diagram	7
6	Project building phases.....	8
6.1	Phase 1 - Provision Azure Resources	8
6.2	Phase 2 - Design ADLS Gen2 Storage Structure	8
6.3	Phase 3 - Azure Databricks Workspace Configuration	8
6.4	Phase 4 – Azure Databricks Notebook Development	8
6.5	Phase 5 – Azure Data Factory Pipeline Development.....	9
7	Project Implementation.....	10
7.1	Azure Databricks Workspace Setup	10
7.1.1	Create Azure Databricks Workspace	10
7.1.2	Create Compute Cluster.....	14
7.2	Configure Azure Databricks with ADLS Gen2	17
7.2.1	Grant ADLS Gen2 Access to Azure Databricks Unity Catalog	17
7.2.2	Create Storage Credential	23
7.2.3	Create External Location	26
7.3	Develop Databricks ETL Notebook.....	32
7.3.1	Import PySpark Libraries	33
7.3.2	Read Silver Layer Data	33
7.3.3	Data Profiling	33
7.3.4	Data Quality Checks	35
7.3.5	Data Cleaning.....	36
7.3.6	Perform Transformation.....	38
7.3.7	Aggregations.....	39
7.3.8	Join MARA and VBAP	39
7.3.9	Validation	40
7.3.10	Write Gold Layer.....	40
7.3.11	Create Delta Table	40
7.3.12	Verification.....	41
7.4	Build Azure Data Factory Pipeline	42

7.4.1	Create Azure Databricks Access Token	42
7.4.2	Create Azure Databricks Linked Service	44
7.4.3	Create Pipeline	46
7.4.4	Add Databricks Notebook Activity	46
7.4.5	Configure the Notebook Activity	47
7.4.6	Execute Pipeline	48
7.4.7	Verification	48
8	Conclusion	50

1 Objective

To design and implement an end-to-end ETL pipeline using Azure Databricks, Azure Data Factory, and Azure Data Lake Storage Gen2 that automates data extraction, transformation, validation, and loading of data from the Silver layer to the Gold layer for analytics and reporting.

2 Technologies Used

Technology	Purpose
Azure Databricks	Data transformation and ETL processing using PySpark
PySpark	Data profiling, cleaning, transformation, aggregation, and joins
Azure Data Factory (ADF)	Pipeline orchestration and notebook execution
Azure Data Lake Storage Gen2 (ADLS Gen2)	Storage for Silver and Gold layer data
Unity Catalog	Governance and management of Delta tables
Delta Lake	Storage format for analytics-ready data
CSV Files	Source and target data format
SQL	Querying and validating the Delta table

3 High Level Business Requirements

1. Requirement 1 – Read Source Data

Read the following source files from the **Silver layer** of Azure Data Lake Storage Gen2 into Azure Databricks Data Frames:

- MARA_ADB.csv (Material Master Data)
- VBAP_ADB.csv (Sales Order Item Data)

2. Requirement 2 – Perform Data Transformation

Perform data profiling, data quality checks, cleansing, transformations, aggregations, and joins to prepare an analytics-ready dataset.

3. Requirement 3 – Load Data to Gold Layer

Write the transformed dataset to the **Gold layer** in Azure Data Lake Storage Gen2 in **CSV** format for downstream consumption.

4. Requirement 4 – Create Analytics Table

Create a **Delta table** in Unity Catalog using the transformed dataset to enable efficient querying and analytics.

5. Requirement 5 – Automate the ETL Process

Create an **Azure Data Factory (ADF) pipeline** to orchestrate the execution of the Azure Databricks notebook and automate the end-to-end ETL process.

4 Solution Architecture

This project implements an end-to-end ETL pipeline using Azure services to transform material master and sales order data into an analytics-ready dataset.

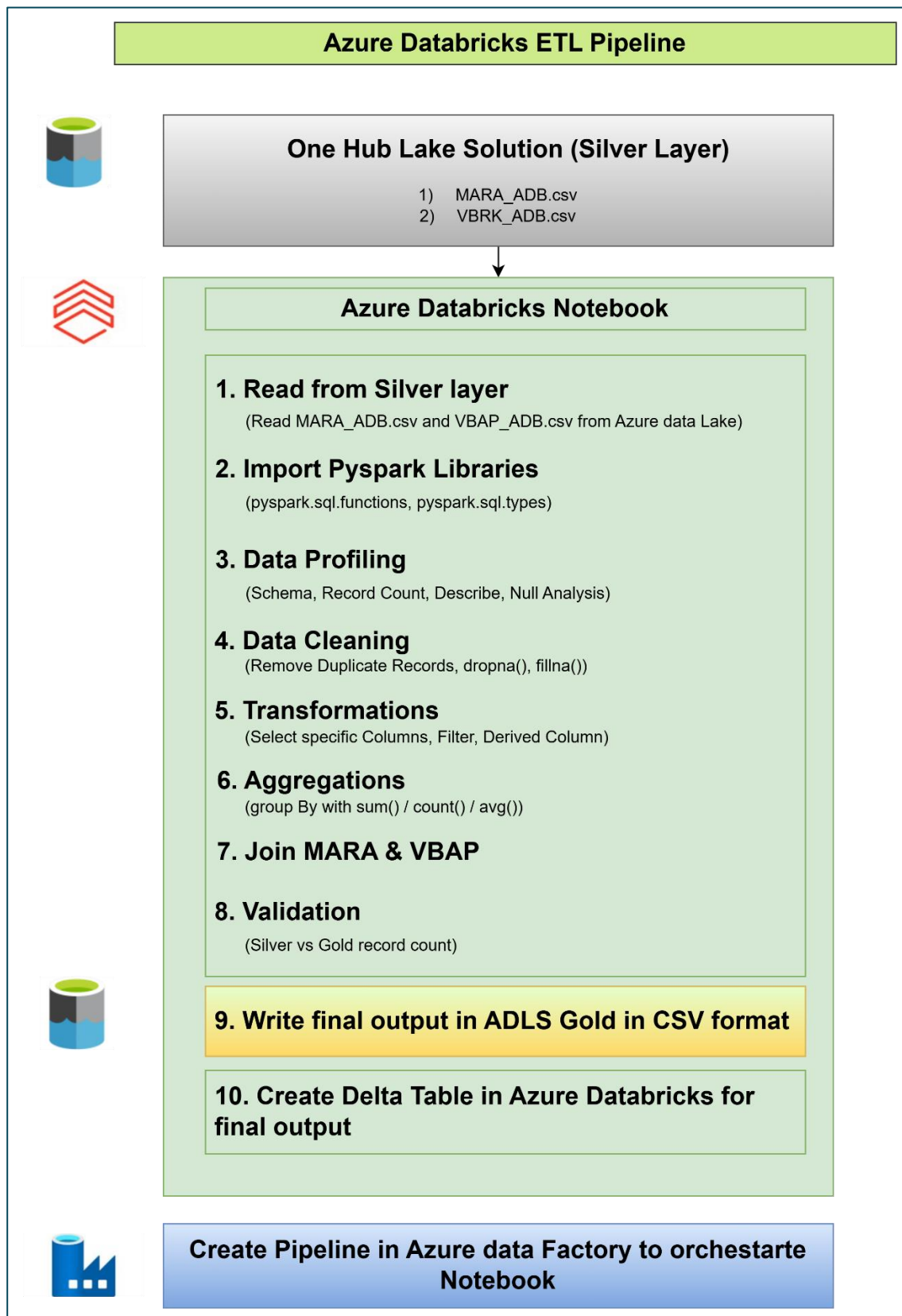
The pipeline begins with source data stored in the **Silver layer** of **Azure Data Lake Storage Gen2 (ADLS Gen2)**. **Azure Data Factory (ADF)** orchestrates the workflow by triggering an **Azure Databricks** notebook.

Within Azure Databricks, the notebook performs the following operations:

- Reads **MARA** (Material Master) and **VBAP** (Sales Order Items) data from the Silver layer.
- Performs data profiling by analyzing the schema, record counts, and summary statistics.
- Applies data quality checks by identifying null values.
- Cleans the data using `dropna()`, `fillna()`, and `dropDuplicates()`.
- Transforms the data by selecting required columns, filtering finished products, and creating derived columns.
- Performs aggregations to summarize sales data using functions such as `SUM`, `COUNT`, and `AVG`.
- Joins the Material Master and Sales Order data using the **Material Number (MATNR)** to create a consolidated Gold dataset.
- Performs validation by comparing source and target record counts to ensure successful data processing.
- Writes the transformed dataset to the **Gold layer** in Azure Data Lake Storage Gen2 in **CSV** format.
- Creates a **Delta table** in **Unity Catalog** to support analytical querying and reporting.

The solution follows the **Medallion Architecture**, where data is progressively refined from the **Silver layer** to the **Gold layer**, resulting in a cleansed, transformed, and analytics-ready dataset.

5 Project Architecture Diagram



6 Project building phases

6.1 Phase 1 - Provision Azure Resources

1. Azure Storage Account (ADLS Gen2)
2. Azure Databricks Workspace
3. Azure Data Factory

6.2 Phase 2 - Design ADLS Gen2 Storage Structure

1. Create the storage container:
 - onehub
2. Create the directory structure:
 - bronze
 - silver
 - gold
3. Configure Azure Databricks access:
 - Grant ADLS Gen2 Storage Account access to Azure Databricks Unity Catalog (Managed Identity)

6.3 Phase 3 - Azure Databricks Workspace Configuration

1. Configure the Azure Databricks workspace.
2. Create a compute cluster.
3. Configure Unity Catalog to access ADLS Gen2 through Storage Credentials and External Locations
4. Generate a Personal Access Token (PAT) for Azure Databricks.
5. Configure an Azure Databricks Linked Service in Azure Data Factory using the Personal Access Token.

6.4 Phase 4 – Azure Databricks Notebook Development

Develop a PySpark notebook to perform the following operations:

1. Read MARA and VBAP data from the Silver layer in ADLS Gen2.
2. Import the required PySpark libraries.
3. Perform data profiling:
 - Schema
 - Record Count
 - Summary Statistics
 - Null Analysis
4. Perform data cleaning:
 - Remove duplicate records
 - Handle null values using dropna()
 - Replace null values using fillna()
5. Perform data transformations:
 - Select required columns
 - Filter finished products
 - Create derived columns
6. Perform aggregations:
 - GROUP BY
 - SUM
 - COUNT

- AVG
- 7. Join MARA and VBAP datasets.
- 8. Validate the transformed data by comparing Silver and Gold record counts.
- 9. Write the transformed data to the Gold layer in ADLS Gen2 in CSV format.
- 10. Create a Delta table in Unity Catalog for analytical querying.

6.5 Phase 5 – Azure Data Factory Pipeline Development

Develop an Azure Data Factory pipeline to:

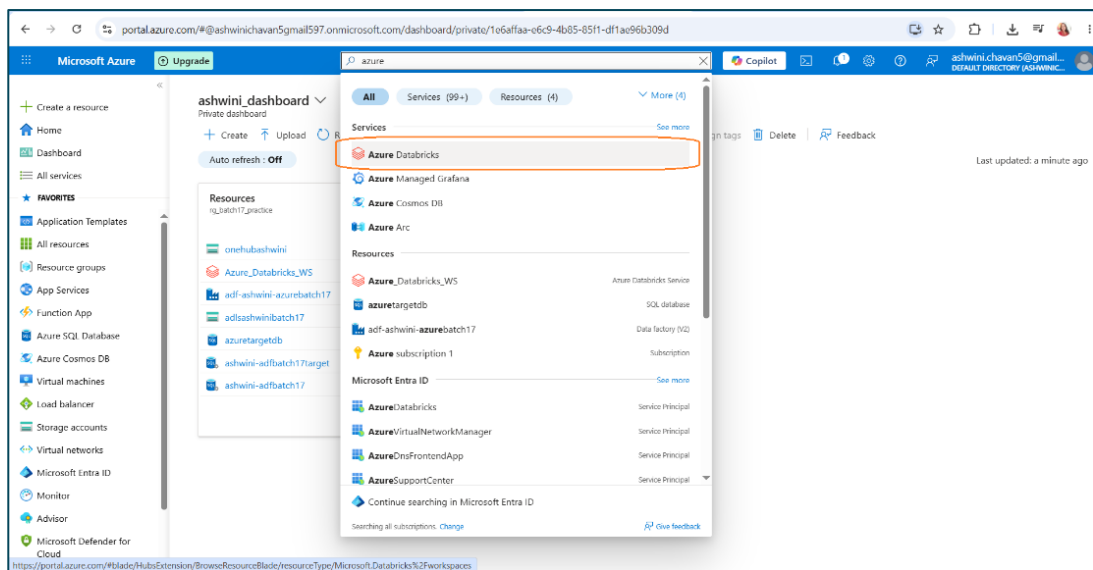
1. Execute the Azure Databricks notebook.
2. Orchestrate the end-to-end ETL process.
3. Monitor and validate successful pipeline execution.

7 Project Implementation

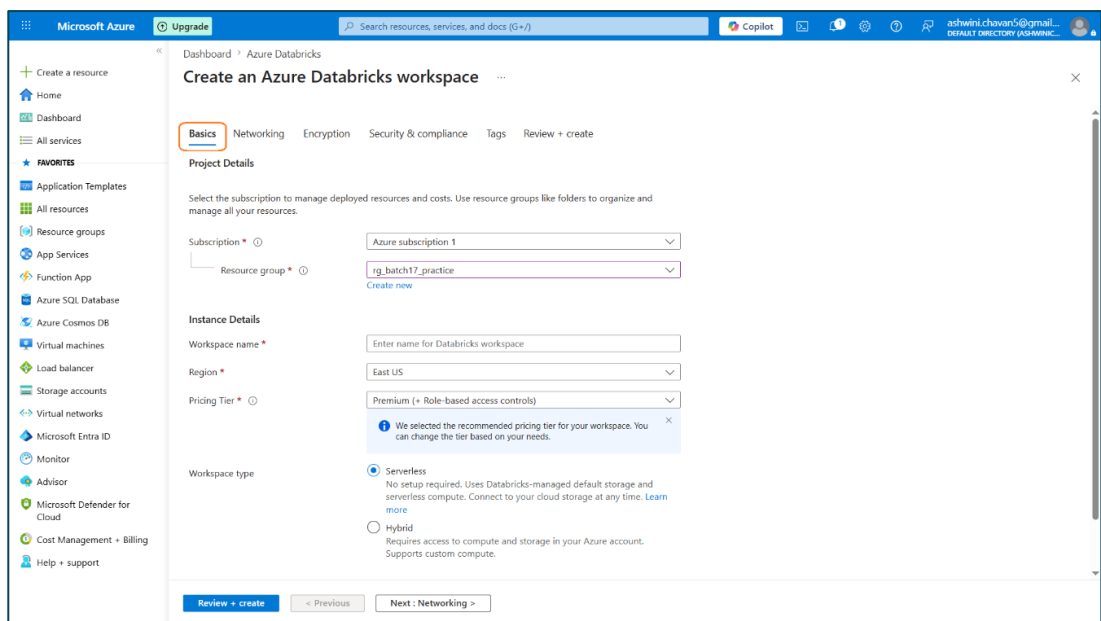
7.1 Azure Databricks Workspace Setup

7.1.1 Create Azure Databricks Workspace

- 1) Search Azure Databricks in Azure portal:
Click on “Azure Databricks” to open the “Create Azure Databricks Workspace screen”:

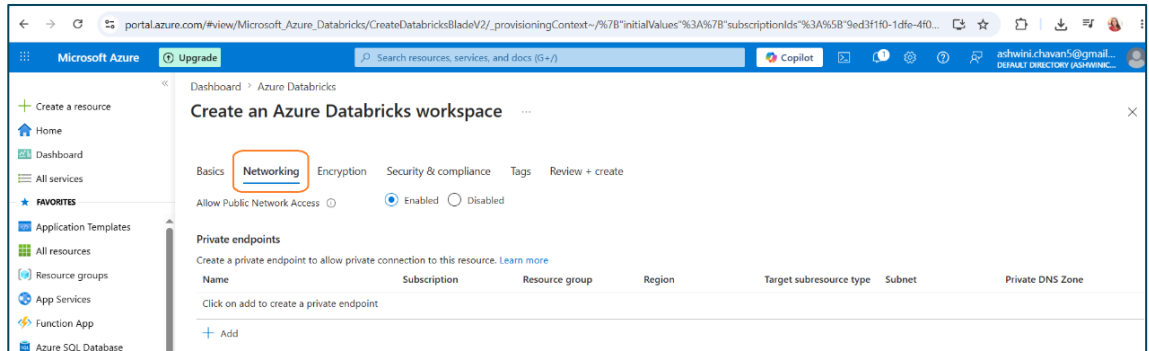


- 2) **Basic Tab:** Set Following values in Basic tab:



Setting	Selected Value	Description
Subscription	Azure Subscription 1	Subscription where the workspace will be created
Resource Group	rg_batch17_practice	Logical container for Azure resources
Workspace Name	adb-ashwini-btach17	Unique name of the Databricks workspace
Region	East US	Same region as other Azure resources to reduce latency
Workspace Type	Serverless	Managed workspace option selected for this project

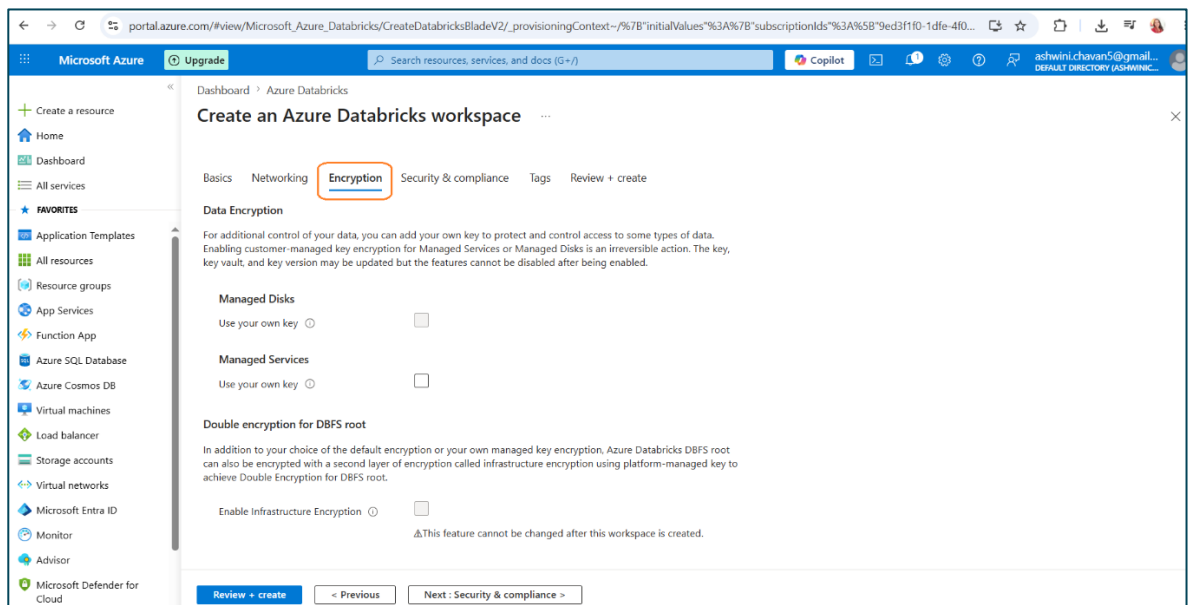
3) Networking Tab: Enable Public Network Access:



Setting	Selected Value	Description
Public Network Access	Enabled	Allows users and integrated Azure services to access the Azure Databricks workspace over the public internet. This option was selected for the project to simplify connectivity and development.

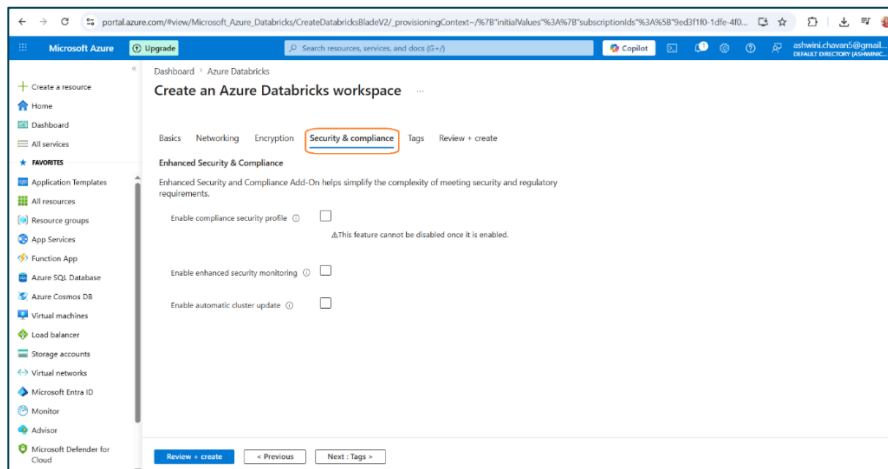
4) Encryption Tab:

No changes were required for this project. The default settings were retained



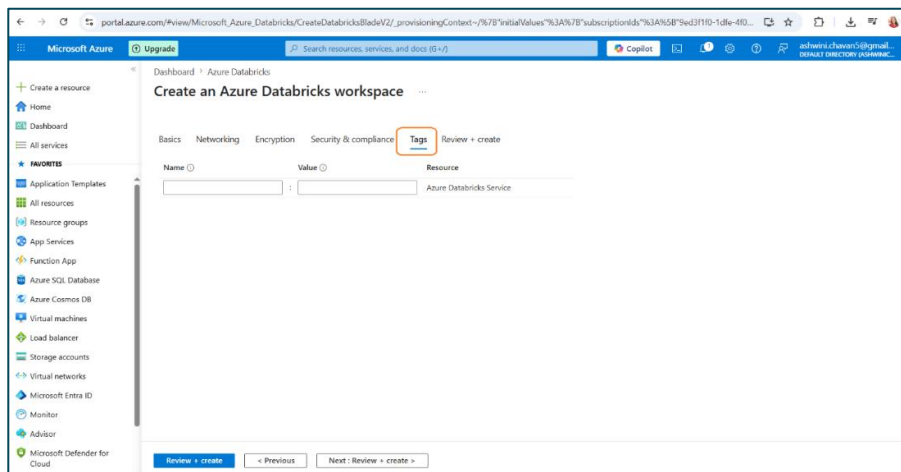
5) Security & Compliance Tab:

No changes were required for this project. The default settings were retained



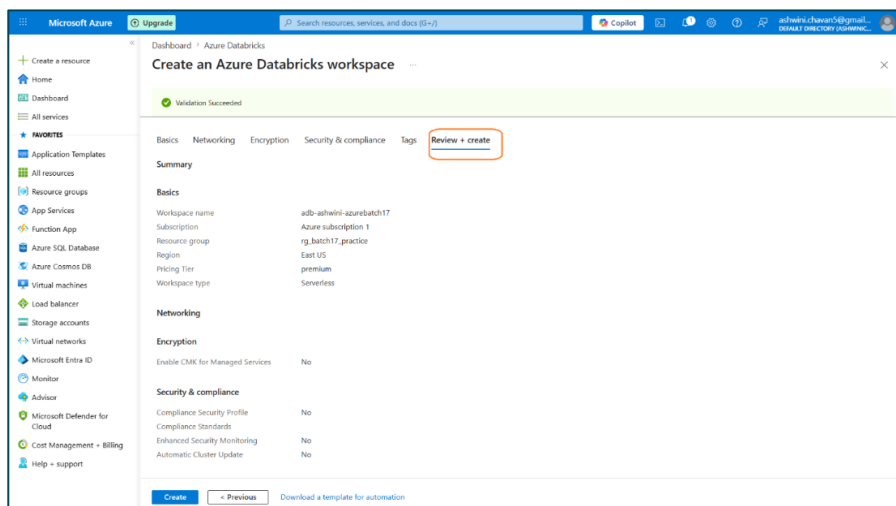
6) Tags Tab: No changes are required:

No changes were required for this project. The default settings were retained



7) Review + Create Tab:

Review all the configuration settings before deploying the Azure Databricks workspace. Click **Create** to deploy the Azure Databricks workspace.:



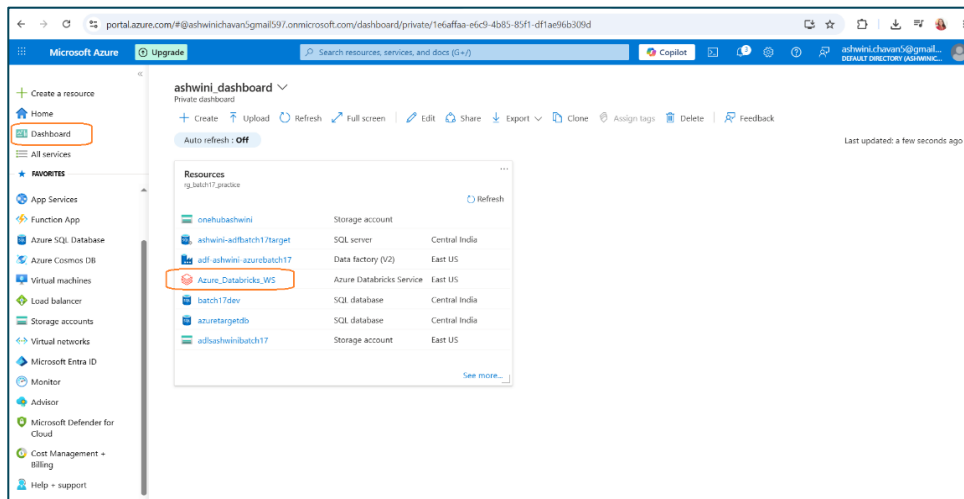
- 8) After the deployment is completed, navigate to the newly created Azure Databricks workspace.

Steps:

- Go to the **Azure Portal**.
- Open **Dashboard**
- Click the newly created **Azure Databricks Workspace**.

Purpose:

Verify that the Azure Databricks workspace has been successfully deployed and is ready for configuration.



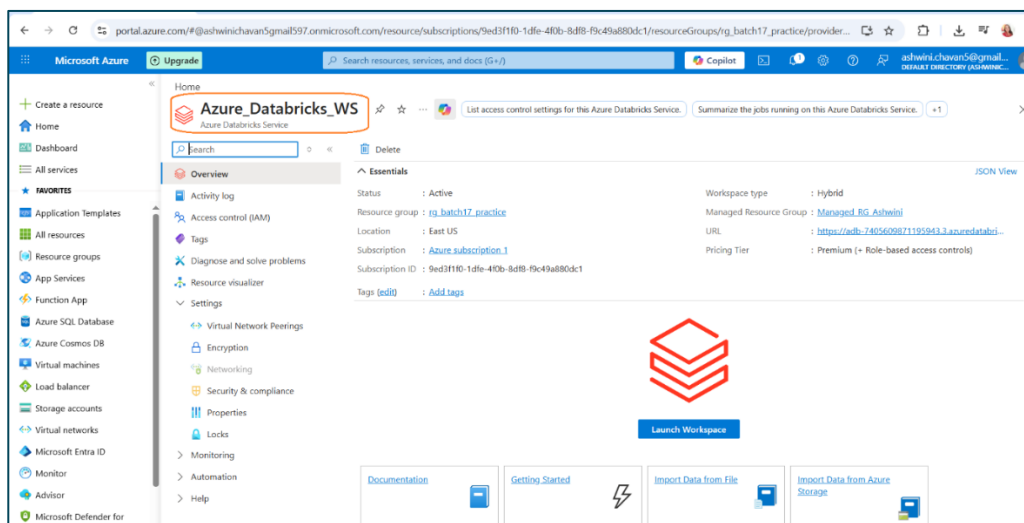
- 9) Review the workspace details displayed on the **Overview** page.

Verify the following details:

- Workspace Name
- Resource Group
- Region
- Workspace URL
- Subscription
- Workspace Status

Purpose:

Verify that the workspace has been successfully provisioned and that all configuration details match the values specified during workspace creation.

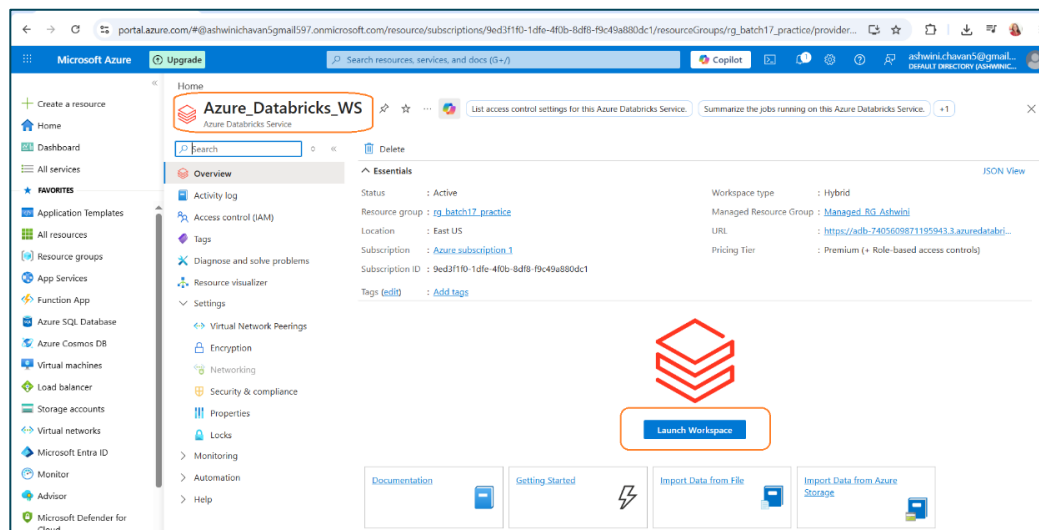


7.1.2 Create Compute Cluster

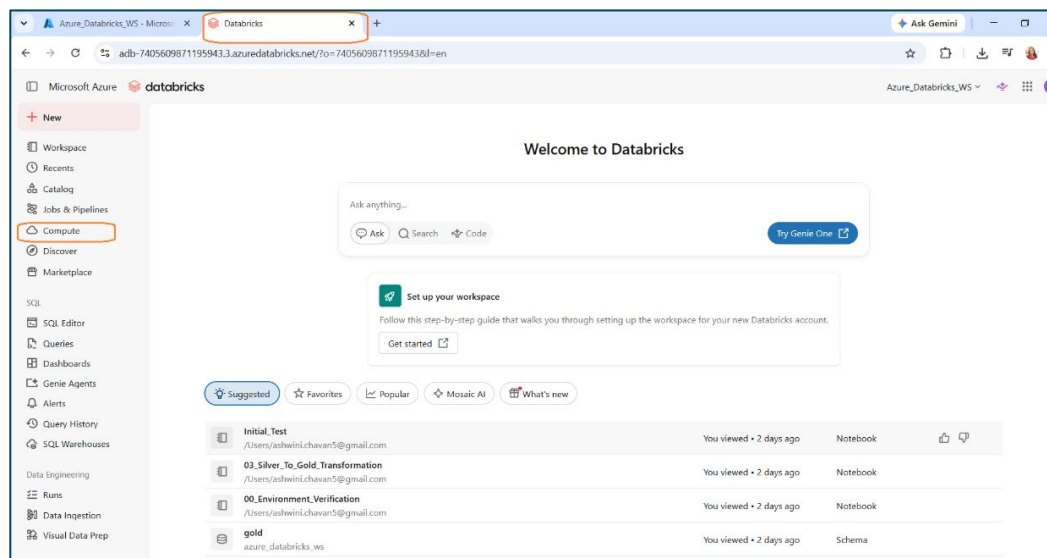
Purpose

Create a compute resource in Azure Databricks to execute the PySpark notebook for data transformation, validation, and loading

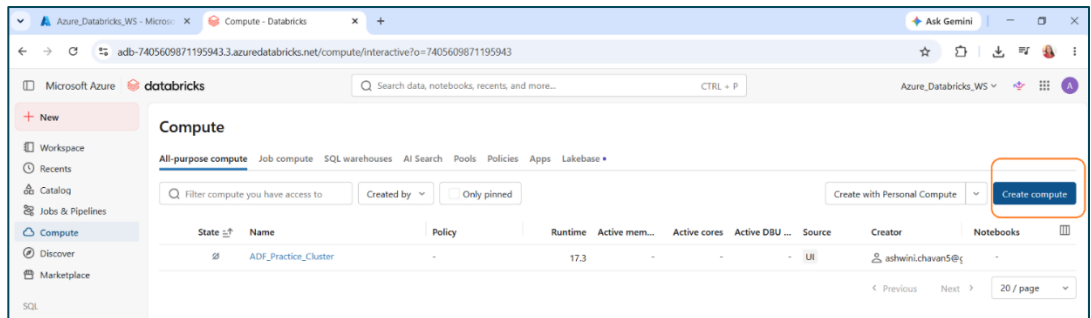
- 1) Launch Azure Databricks Workspace
Open the Azure Databricks workspace from the Azure Portal:



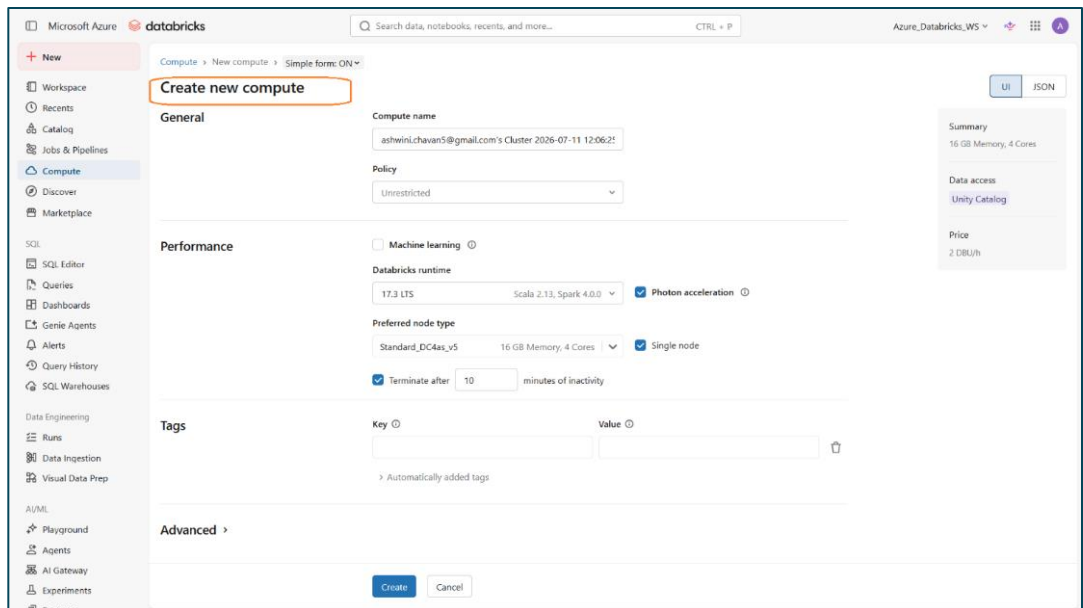
- 2) Open the Compute Page
From the left navigation pane, click **Compute**:



- 3) Create a New Compute
Click the “Create Compute” button:



Following screen will be displayed:



- 4) Configure the Compute

Provide the following configuration values:

Section	Setting	Value
General	Compute Name	ADF_Practice_Cluster
General	Policy	Unrestricted
Performance	Machine Learning	Disabled
Performance	Databricks Runtime	17.3 LTS (Apache Spark 4.0.0, Scala 2.13)
Performance	Photon Acceleration	Enabled
Performance	Preferred Node Type	Standard_F4ads_v7 (16 GB Memory, 4 Cores)
Performance	Single Node	Enabled
Performance	Auto Termination	15 minutes of inactivity
Tags	Tags	No custom tags were configured.
Advanced	Access Mode	Manual - Standard

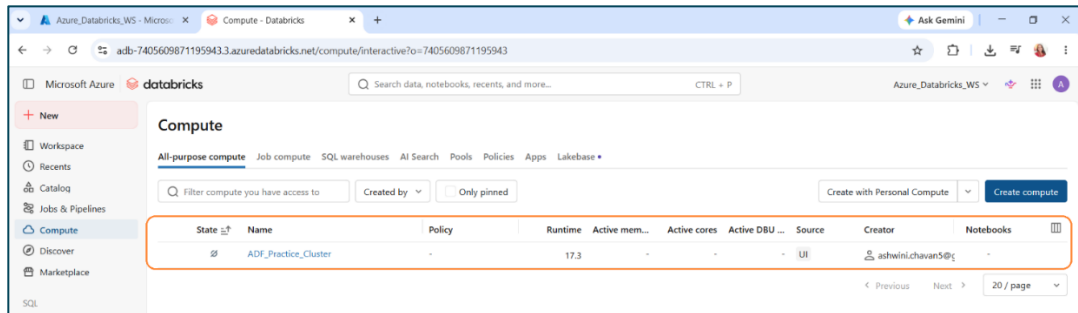
Click **Create** to provision the compute.

5) Verify Compute Creation

After the compute is created, it appears in the **Compute** list.

Verify:

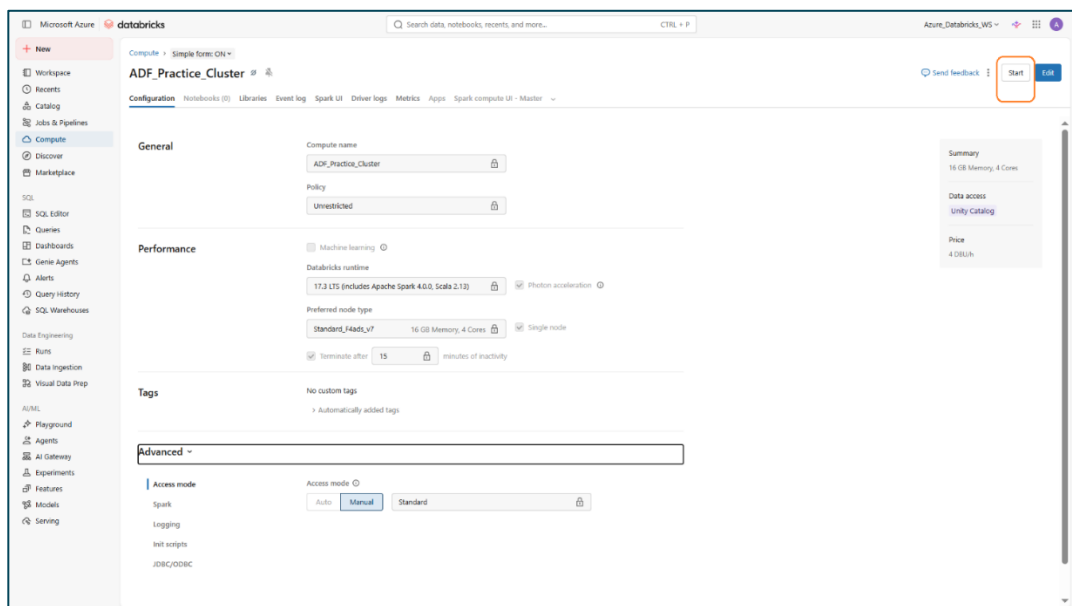
- Compute Name
- Runtime Version
- State



6) Start the Compute

Open **ADF_Practice_Cluster** and click **Start** if the compute is not already running.

Wait until the status changes to **Running**.



7) Important Configuration Notes

- **Databricks Runtime 17.3 LTS** was selected because it provides a stable Long-Term Support (LTS) runtime with Apache Spark 4.0.0.
- **Photon Acceleration** was enabled to improve query and data processing performance.
- **Single Node** compute was sufficient for this project since the dataset is relatively small and intended for demonstration purposes.
- **Auto Termination** was configured to stop the compute after **15 minutes** of inactivity, helping reduce unnecessary Azure costs.
- **Manual Standard** access mode was used to provide standard access for notebook execution.

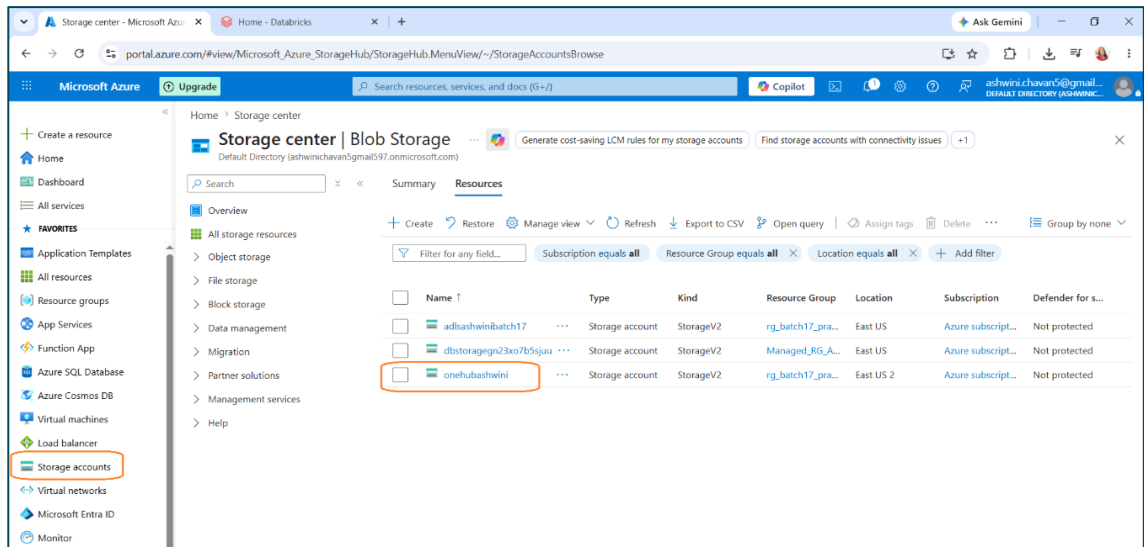
7.2 Configure Azure Databricks with ADLS Gen2

7.2.1 Grant ADLS Gen2 Access to Azure Databricks Unity Catalog

Grant the **Azure Databricks Unity Catalog Managed Identity** the required permissions to access data stored in Azure Data Lake Storage Gen2.

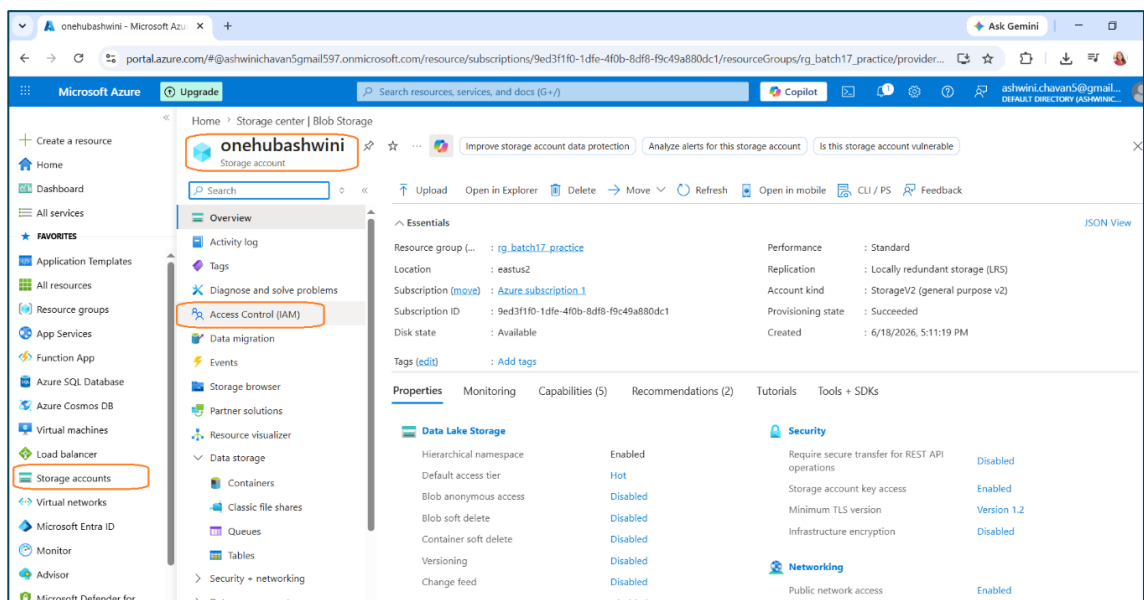
1) Open the Storage Account:

Navigate to the Azure Storage Account that will be used by Azure Databricks.



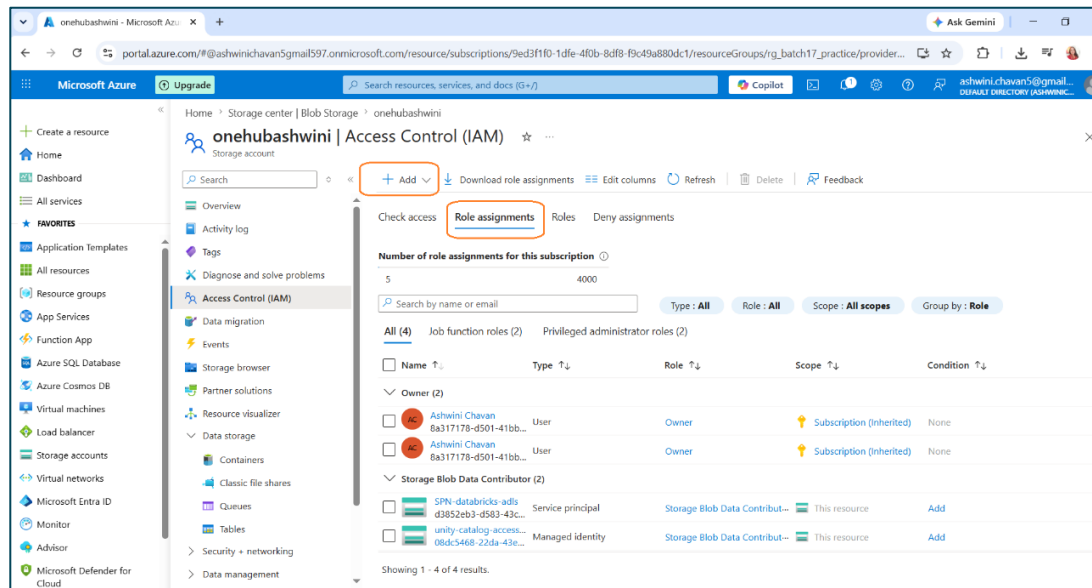
2) Open Access Control (IAM):

From the left navigation pane, select **Access Control (IAM)**.



3) Add a New Role Assignment:

Click: + Add → Add Role Assignment

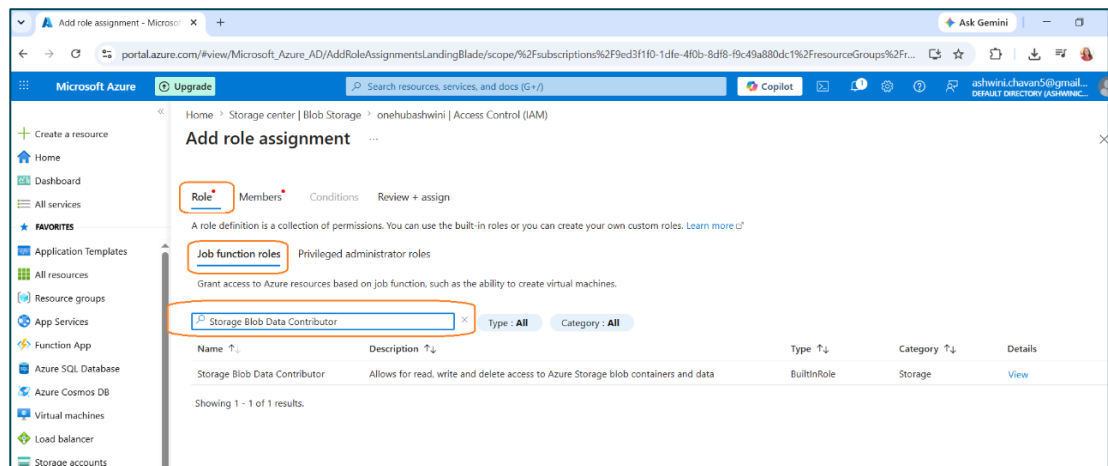


4) Select the Role:

Search for and select the following role:

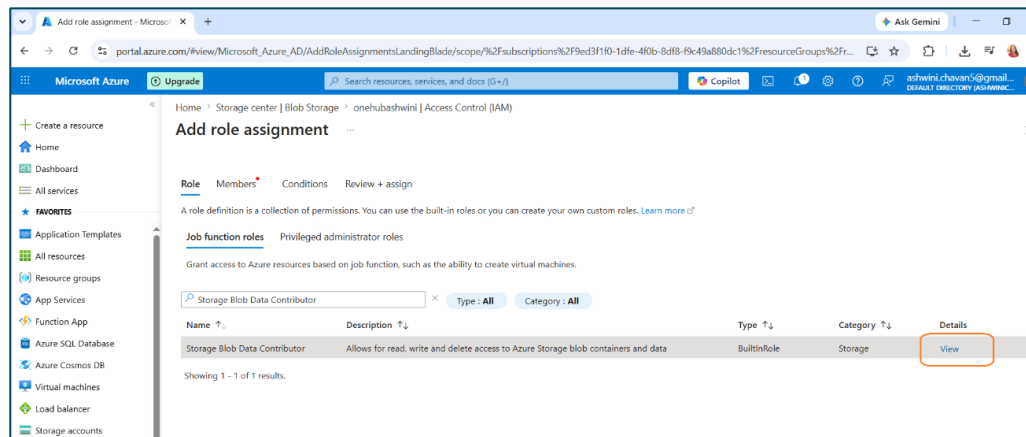
Storage Blob Data Contributor

This role grants the managed identity permission to read, write, and manage blob data within the storage account.

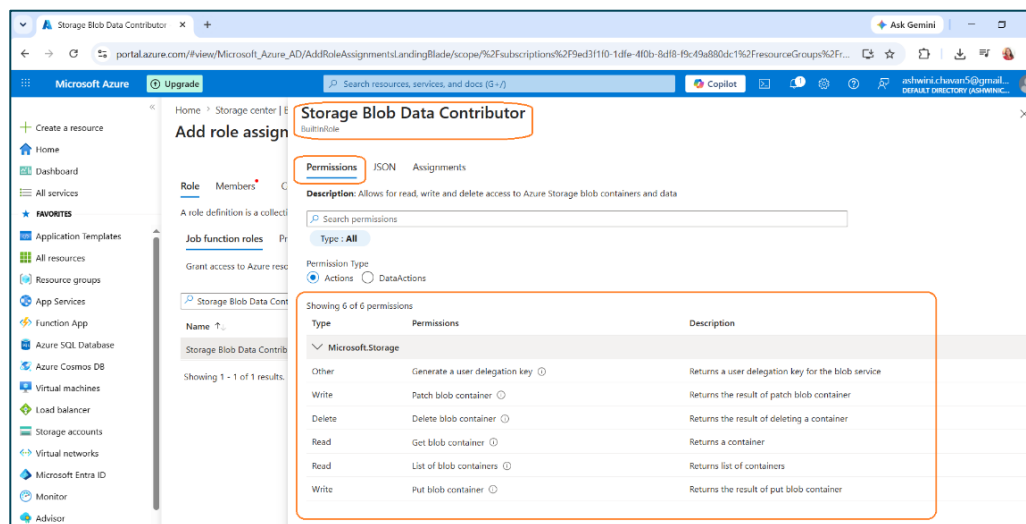


5) Review Permissions:

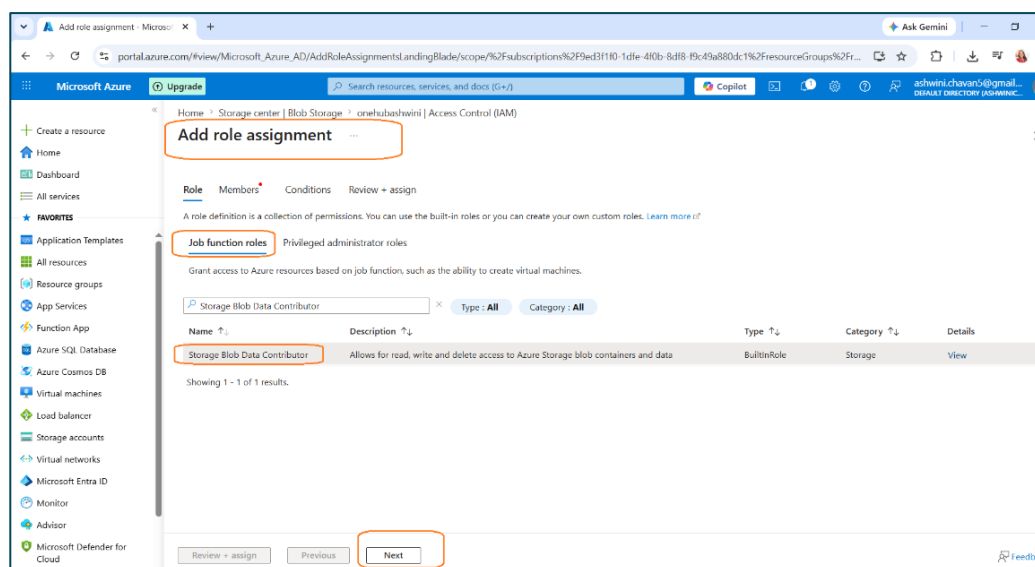
Review the permissions associated with the selected role.



Review permissions and close the permission tab:



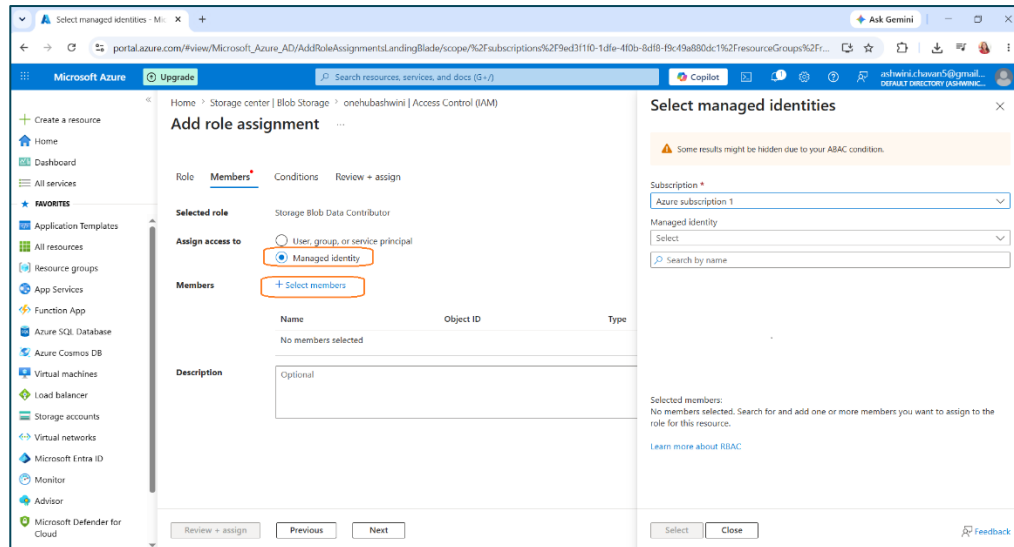
Click on Next:



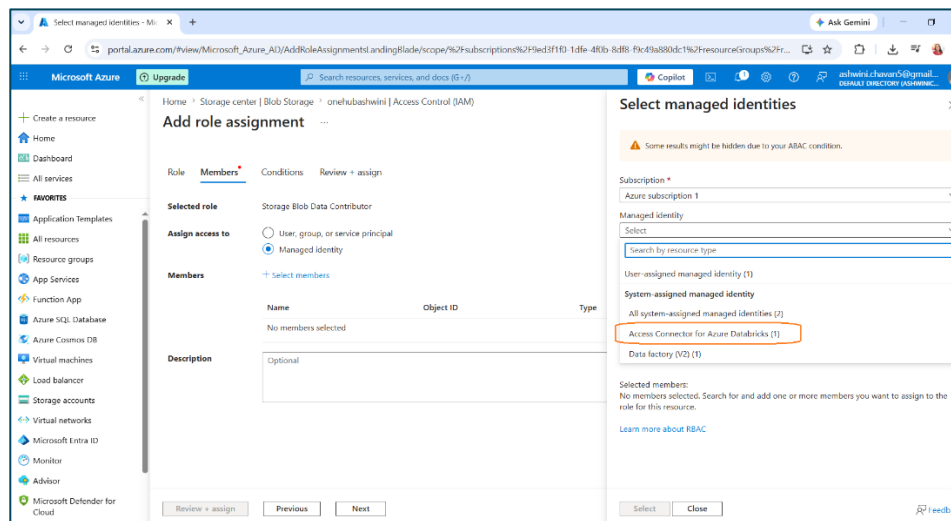
6) Select the Managed Identity

Under Members tab:

- Select **Managed Identity**
- Click **Select Members**



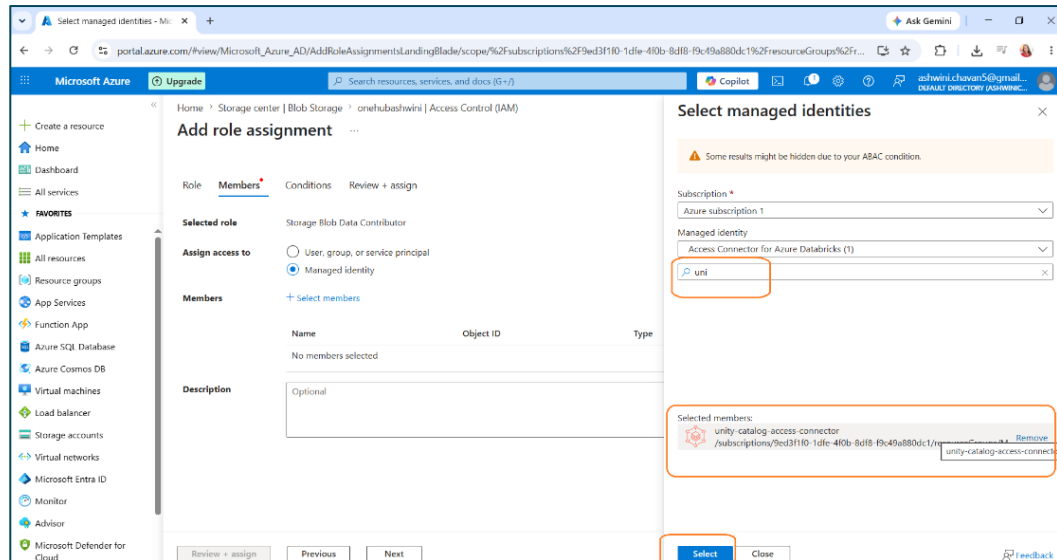
7) In Managed Identity dropdown select “Access Connector for Azure Databricks”:



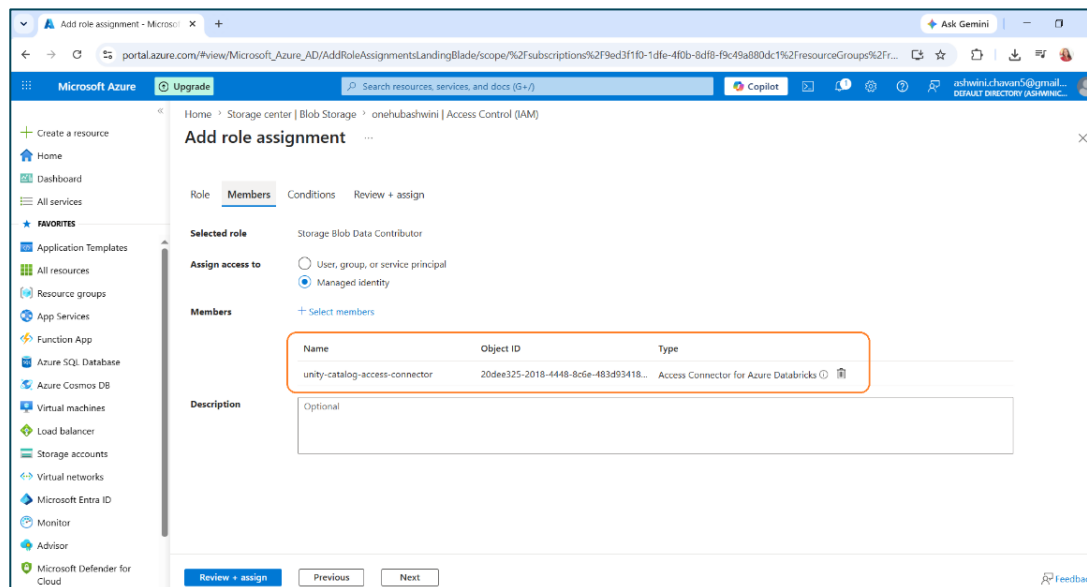
8) Select Unity Catalog:

Search For Unity Catalog and click on “Select” button:

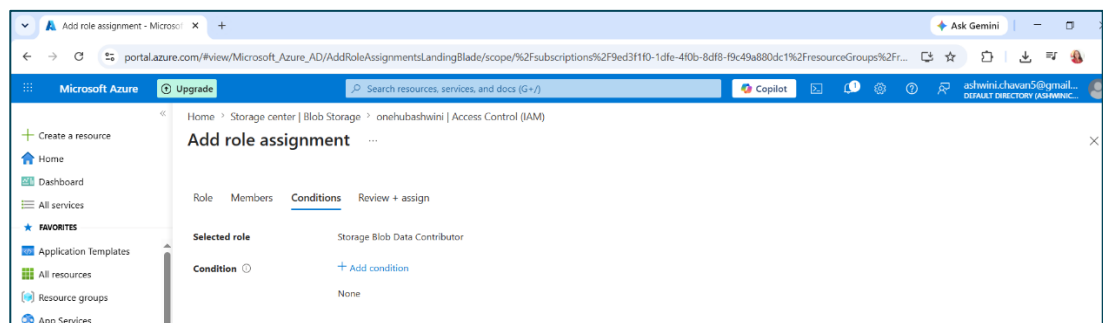
Note: The **Unity Catalog Managed Identity** becomes available only after the Azure Databricks workspace and Unity Catalog have been configured.



Once selected, it will be displayed in a grid:

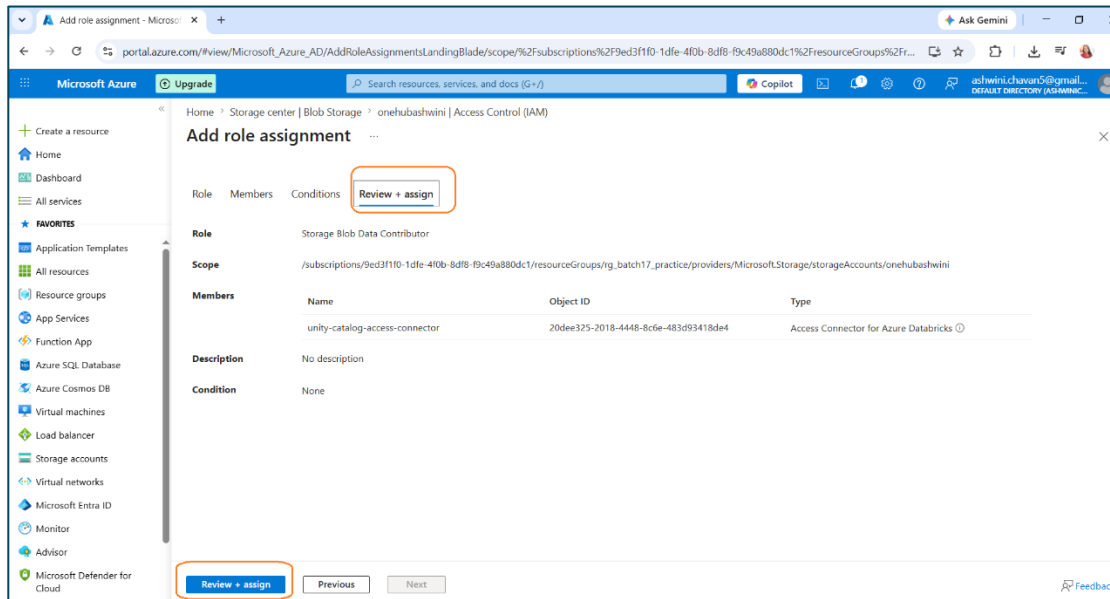


9) Condition tab - No Changes are required for this project:



10) Review and Assign

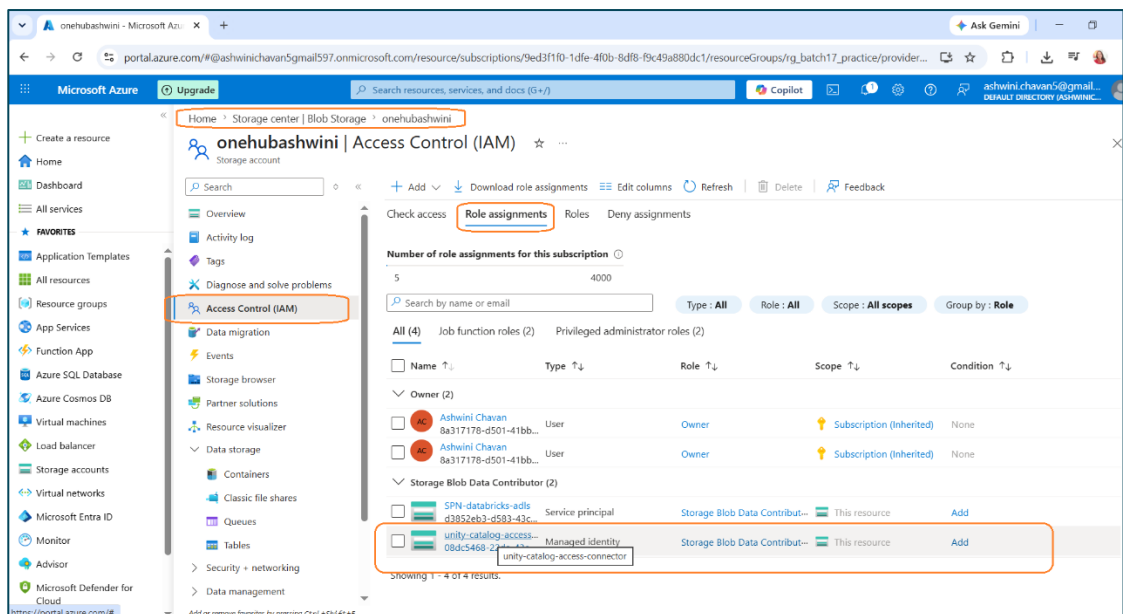
Review the configuration.
Click **Review + Assign**.



11) Verify the Role Assignment

Verify that the following role assignment has been created successfully:

- Name: Unity-catalog-access-connector
- Role: Storage Blob Data Contributor



7.2.2 Create Storage Credential

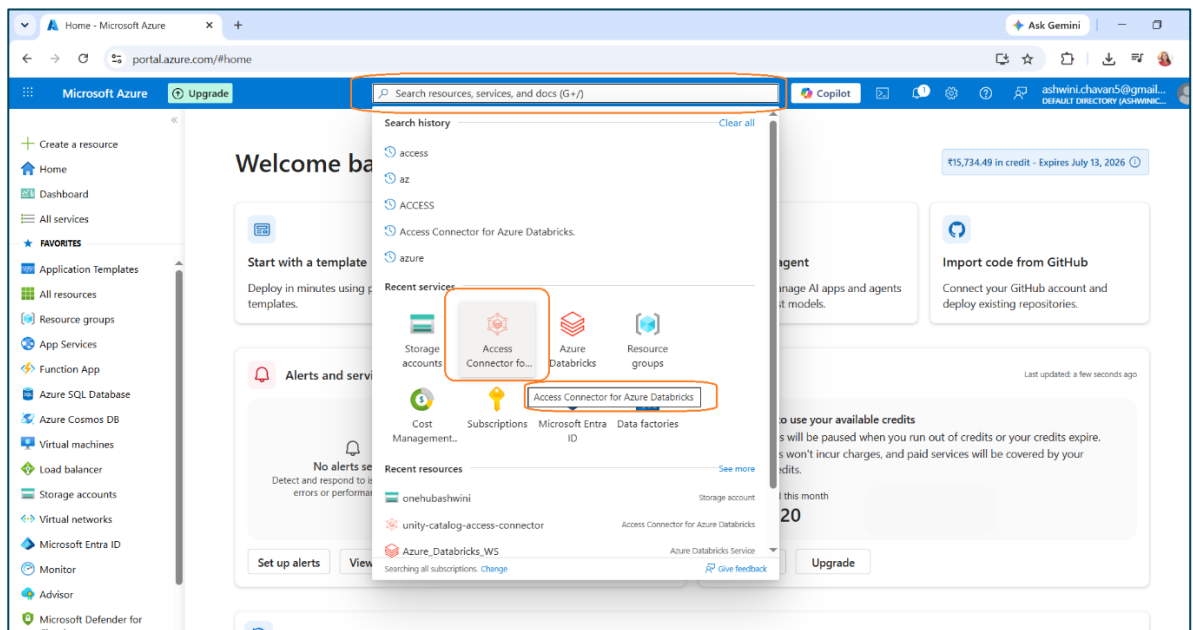
Purpose

Create a Storage Credential in Unity Catalog using the Azure Databricks Access Connector. The Storage Credential enables Unity Catalog to securely access data stored in Azure Data Lake Storage Gen2 using Azure Managed Identity.

1) Open the Azure Databricks Access Connector

In the Azure Portal:

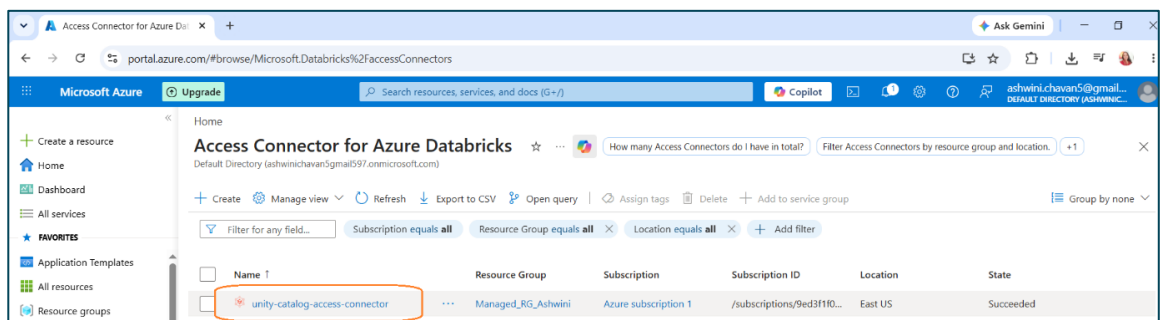
- Search for **Access Connector for Azure Databricks**.
- Select the previously created Access Connector.



On click, the selected “unity-catalog-access-connector” will be displayed in a grid:

2) Open the Access Connector

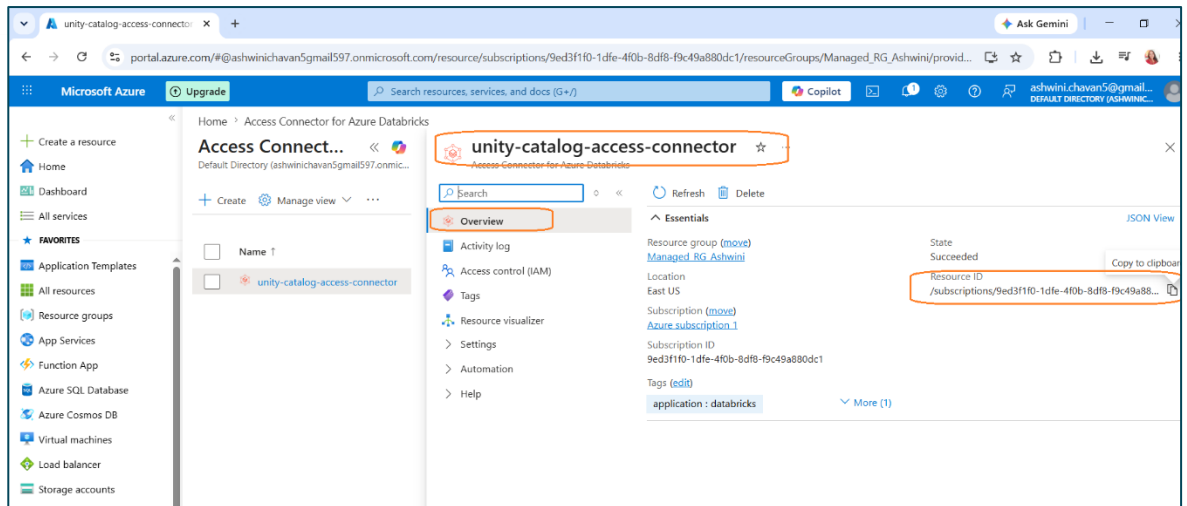
Click the “Unity Catalog access connector”:



3) Copy the Resource ID

From the **Overview** page, copy the **Resource ID**.

This Resource ID will be used while creating the Storage Credential in Unity Catalog.



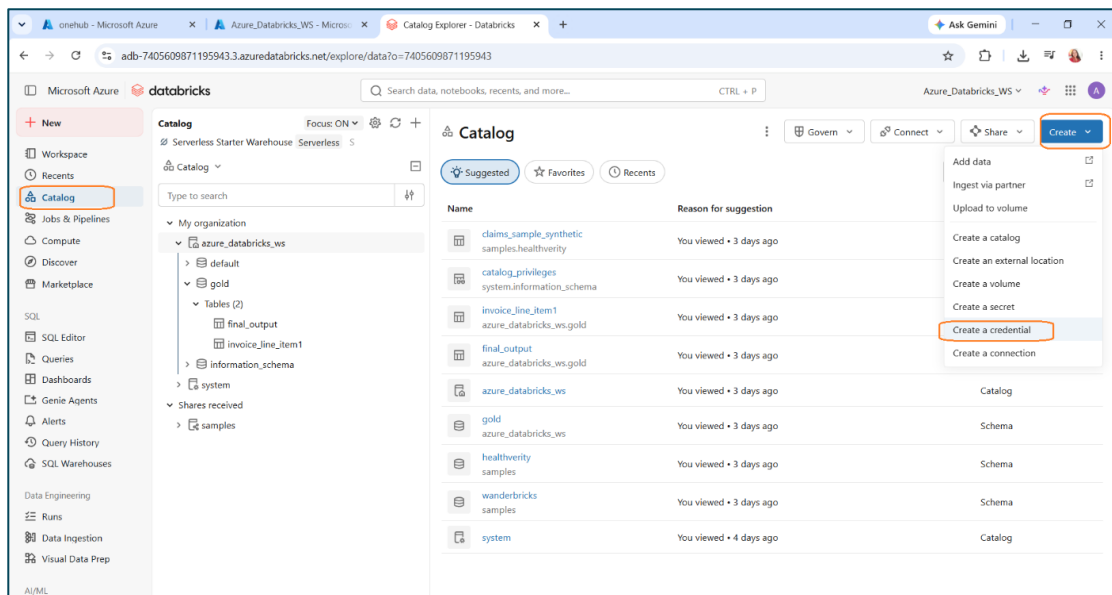
/subscriptions/9ed31f0-1dfe-4f0b-8df8-f9c49a880dc1/resourceGroups/Managed_RG_Ashwini/providers/Microsoft.Databricks/accessConnectors/unity-catalog-access-connector

4) Open Unity Catalog

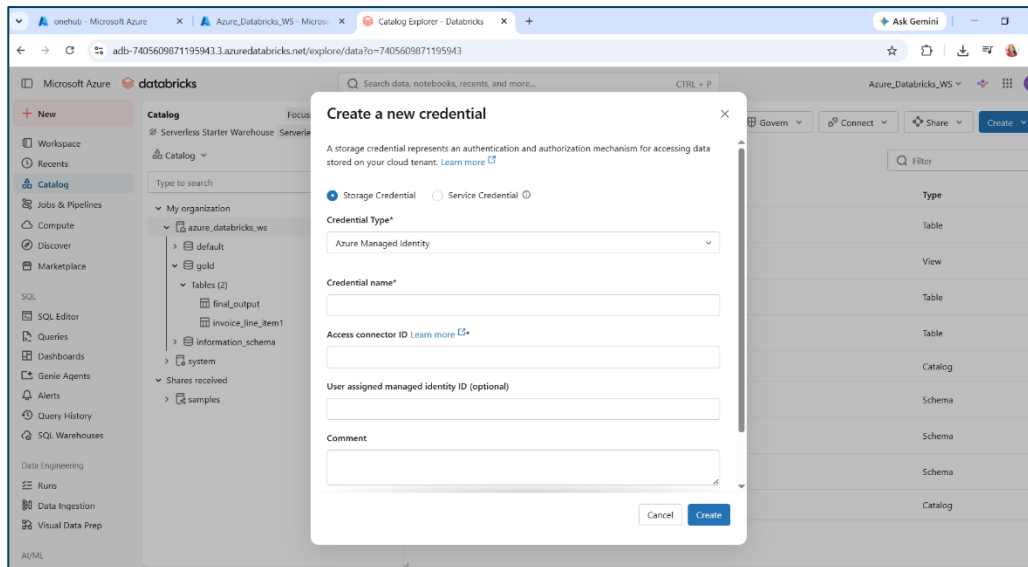
Open Azure Databricks Workspace

Navigate to:

Catalog → Create → Create a Credentials



“Create new credentials” screen will be displayed as shown below:



5) Create the Storage Credential

Enter the following values:

Field	Value
Credential Type	Azure Managed Identity
Credential Name	adf_integration_token
Access Connector ID	Resource ID copied in Step 3 /subscriptions/9ed3f1f0-1dfe-4f0b-8df8-f9c49a880dc1/resourceGroups/Managed_RG_Ashwini/providers/Microsoft.Databricks/accessConnectors/unity-catalog-access-connector
User assigned managed identity ID	Leave blank
Comment	Optional (e.g., Storage Credential for ADLS Gen2)

Click **Create**.

7.2.3 Create External Location

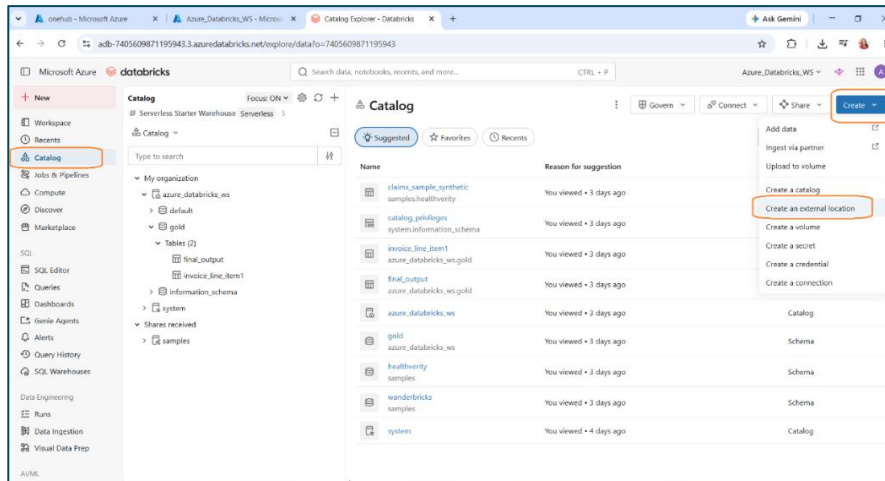
Purpose

Create an External Location in Unity Catalog to enable Azure Databricks to securely access data stored in Azure Data Lake Storage Gen2 using the previously created Storage Credential.

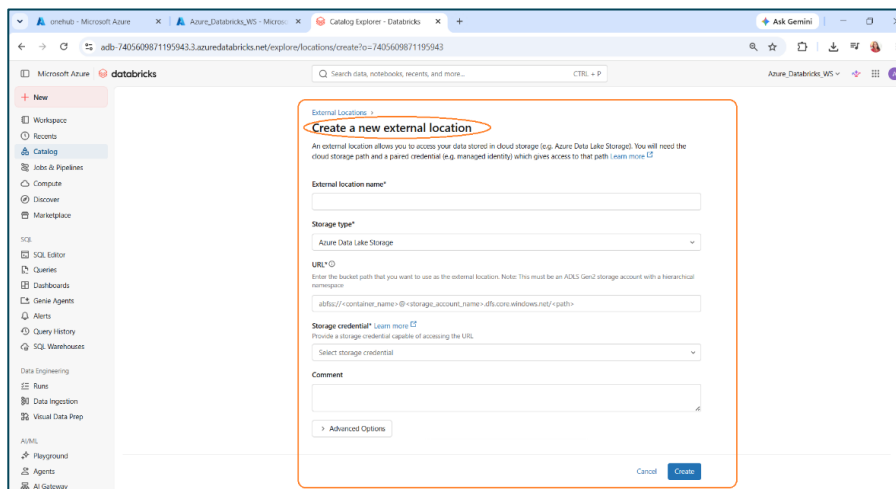
1) Open External Location

Navigate to:

Azure Databricks Workspace → Catalog → Create → Create an external location



Following window will be displayed to create “External Location”:



2) Specify the External Location Details

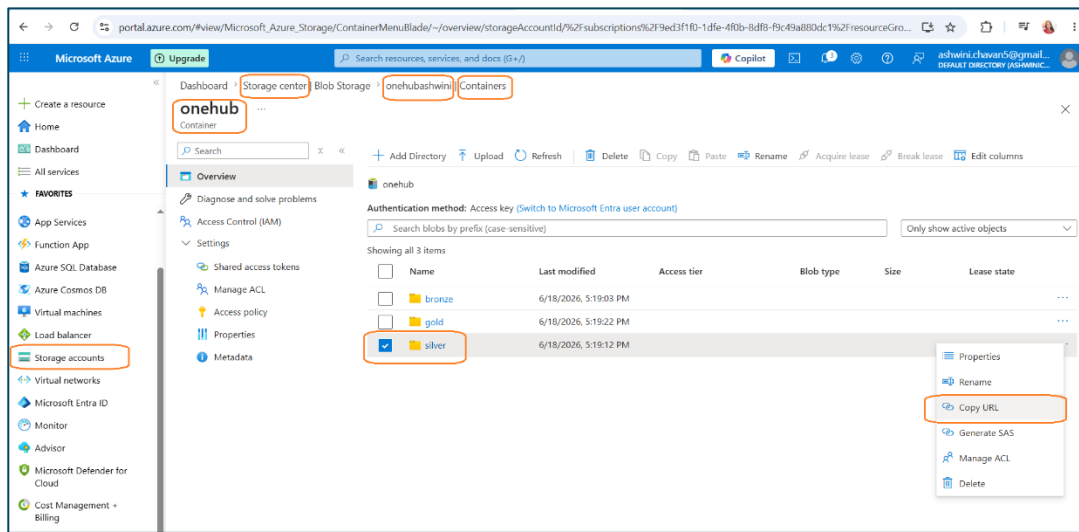
Provide the following information:

- **Name:** onehub_location
- **Storage Type:** Azure Data Lake Storage
- **URL:** abfss://onehub@onehubashwini.dfs.core.windows.net/
- **Storage Credential:** Enter the newly created credential adf_integration_token (catalog → Create credentials)
- **Comment:** (optional) Leave it blank.

Click on **Create**.

3) Understanding the External Location URL

The External Location URL is derived from the Azure Storage Account container URL.



For example, the container URL is:

<https://onehubbashwini.blob.core.windows.net/onehub/>

Unity Catalog requires the **ABFSS** protocol instead of HTTPS, because it communicates with Azure Data Lake Storage Gen2 through the secure Azure Blob File System driver rather than the HTTPS endpoint.

Therefore, convert the URL as follows:

HTTPS URL	https://onehubbashwini.blob.core.windows.net/onehub/
ABFSS URL	abfss://onehub@onehubbashwini.dfs.core.windows.net/

Note:

- Storage Account = **onehubbashwini**
- Container = **onehub**

The External Location is configured at the **container level**, allowing access to all folders within the container, including:

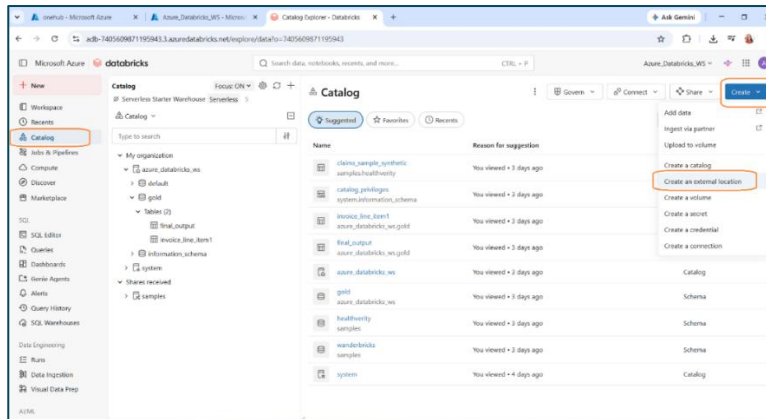
- bronze/
- silver/
- gold/

Configuring the External Location at the container level allows a single Storage Credential and External Location to be reused across all Medallion layers, simplifying administration.

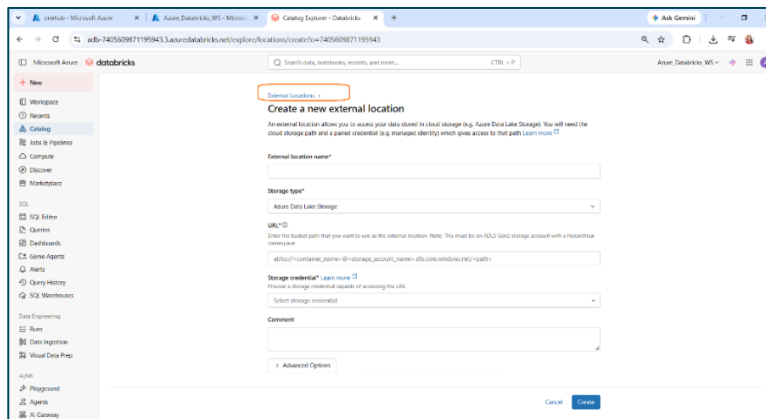
4) Verify the External Location

Navigate to:

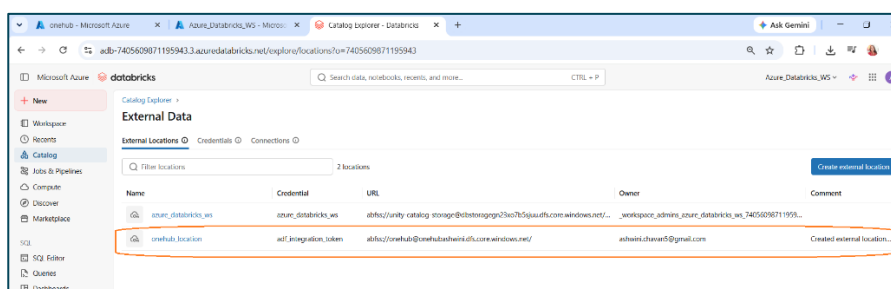
Azure Databricks Workspace → Catalog → Create → Create an external location



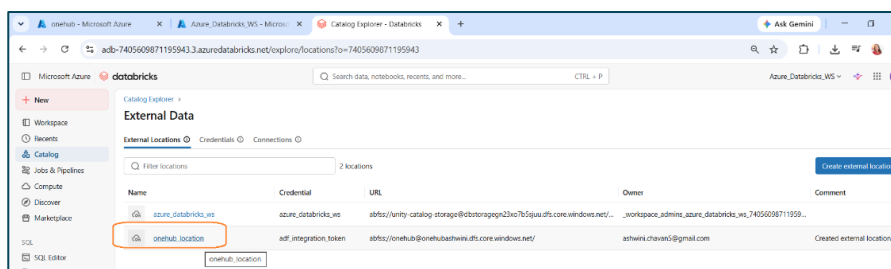
- Click on External Locations link to view newly created external Location:



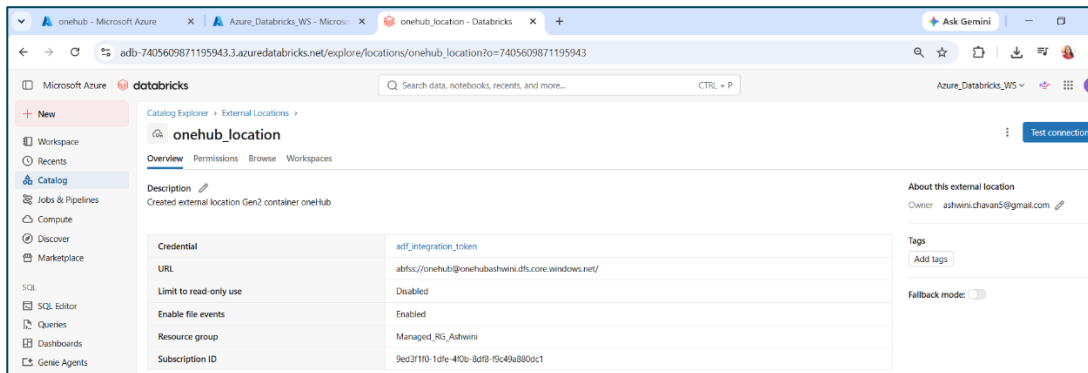
- Following grid will be displayed:



- Click on "onehub_location", to view details:

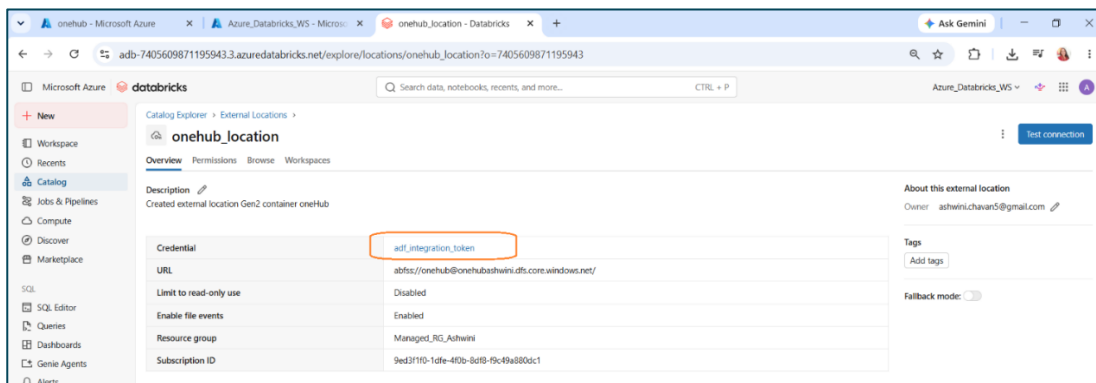


- Verify the following:
 - Storage Credential (adf_integration_token)
 - URL
 - Resource group
 - Subscription ID



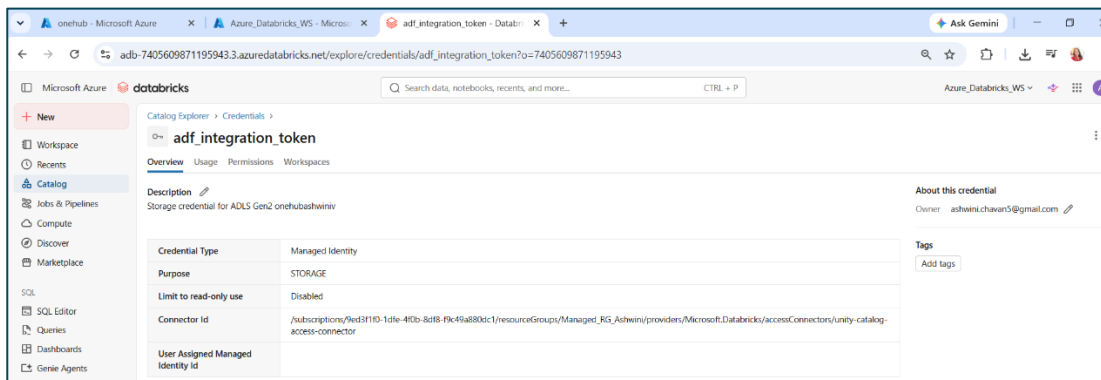
5) Verify the Storage credentials:

Under onehub_location → Click on credentials to view credential details:



Verify:

- Credential Type: should be Managed Identity
- Purpose: Should be storage



6) Verify the Storage Credential using SQL:

- Open a new notebook and execute:
%sql
SHOW STORAGE CREDENTIALS;
- Verify that the Storage Credential **adf_integration_token** is displayed.

The screenshot shows the Databricks interface with a notebook titled '00_Environment_Verification'. The SQL command `%sql SHOW STORAGE CREDENTIALS;` has been executed. The result is a table with two rows:

name	comment
adf_integration_token	Storage credential for ADLS Gen2 onehubashwiniv
azure_databricks_ws	null

The first row, containing the storage credential information, is highlighted with an orange box.

7) Verify External Location using SQL:

- Execute:
%sql
SHOW EXTERNAL LOCATIONS;
- Verify that **onehub_location** is listed.

The screenshot shows the Databricks interface with a notebook titled '00_Environment_Verification'. The SQL command `%sql SHOW EXTERNAL LOCATIONS;` has been executed. The result is a table with two rows:

name	url	comment
azure_databricks_ws	abfss://unity-catalog-storage@dbstorageeg23xo7b5gjuu.dfs.core.windows.net/7405609871195943	null
onehub_location	abfss://onehub@onehubashwiniv.dfs.core.windows.net/	Created external location Gen2 container onehub

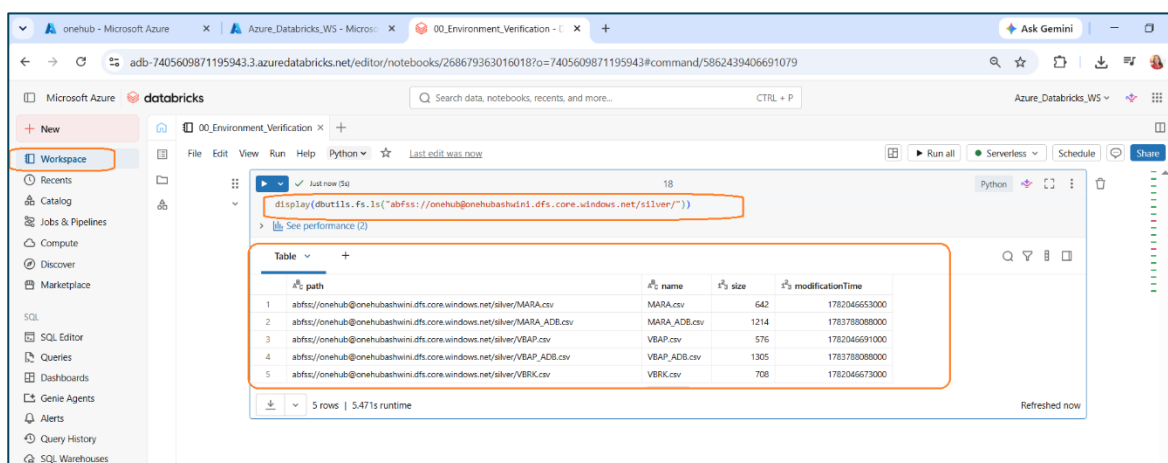
The second row, representing the 'onehub_location', is highlighted with an orange box.

8) Verify Access to ADLS Gen2

- Execute

```
display(
  dbutils.fs.ls(
    "abfss://onehub@onehubashwini.dfs.core.windows.net/silver/"
  )
)
```

- Verify that the folders and CSV files in the Silver layer are displayed successfully.



The screenshot shows the Databricks workspace interface. The command executed is:

```
display(dbutils.fs.ls("abfss://onehub@onehubashwini.dfs.core.windows.net/silver/"))
```

The output is a table with 5 rows, showing the files and folders in the Silver layer:

	path	name	size	modificationTime
1	abfss://onehub@onehubashwini.dfs.core.windows.net/silver/MARA.csv	MARA.csv	642	1782046653000
2	abfss://onehub@onehubashwini.dfs.core.windows.net/silver/MARA_ADE.csv	MARA_ADE.csv	1214	1783788088000
3	abfss://onehub@onehubashwini.dfs.core.windows.net/silver/VBAP.csv	VBAP.csv	576	1782046691000
4	abfss://onehub@onehubashwini.dfs.core.windows.net/silver/VBAP_ADE.csv	VBAP_ADE.csv	1305	1783788088000
5	abfss://onehub@onehubashwini.dfs.core.windows.net/silver/VBRK.csv	VBRK.csv	708	1782046673000

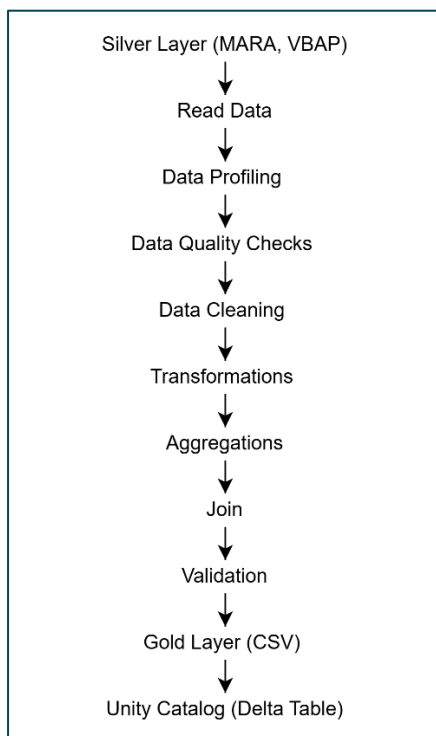
The table indicates 5 rows and a runtime of 5.471s. The status is "Refreshed now".

7.3 Develop Databricks ETL Notebook

Purpose

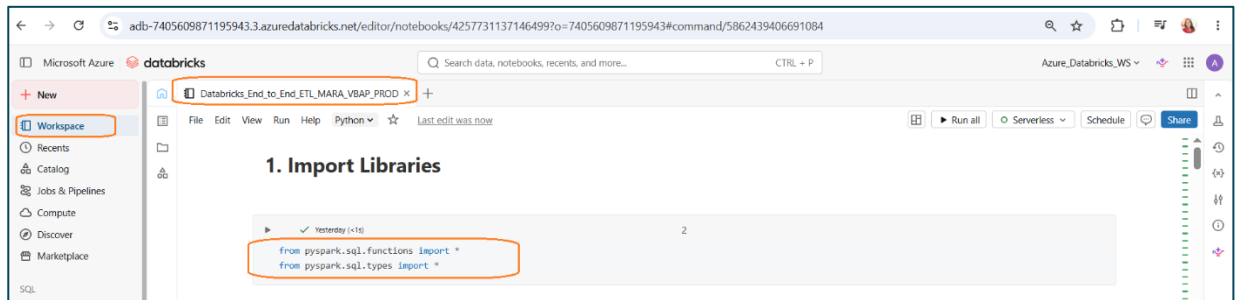
Develop a PySpark notebook to read data from the Silver layer, perform data quality checks and transformations, generate business summaries, and load the final dataset into the Gold layer and Unity Catalog.

Workflow:



7.3.1 Import PySpark Libraries

Import the required PySpark libraries for DataFrame operations, transformations, aggregations, and data type handling.



7.3.2 Read Silver Layer Data

Read the MARA and VBAP source files from the Silver layer of Azure Data Lake Storage Gen2 into PySpark DataFrames.



7.3.3 Data Profiling

Perform initial analysis of the source datasets to understand their structure and contents.

The following profiling operations are performed:

- Schema
- Record Count
- Summary Statistics (describe())

1) Schema

a. Schema

```
mar_a_df.printSchema()  
vbap_df.printSchema()
```

2) Record count

b. Record Count

```
mar_a_count = mar_a_df.count()  
vbap_count = vbap_df.count()  
print("MAR_A :", mar_a_count)  
print("VBAP :", vbap_count)
```

> [See performance \(2\)](#)

MAR_A : 25
VBAP : 50

3) Describe

c. Describe

```
display(mar_a_df.describe())  
display(vbap_df.describe())
```

> [See performance \(2\)](#)

7.3.4 Data Quality Checks

Analyze the datasets to identify potential data quality issues before processing.

The notebook performs:

- Null value analysis

1) Null Analysis

d. Null Analysis

```
display(
  mara_df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in mara_df.columns
  ])
)

display(
  vbap_df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in vbap_df.columns
  ])
)
```

7.3.5 Data Cleaning

Clean the source data to improve its quality before transformation.

The following operations are performed:

- Remove duplicate records
- Remove records containing null values (dropna())
- Replace null values (fillna())
- Verify the cleansed dataset

1) Remove Duplicates:

a. Remove Duplicate Records

✓ Yesterday (<1s) 21

```
mara_clean = mara_df.dropDuplicates()

vbap_clean = vbap_df.dropDuplicates()
```

> mara_clean: pyspark.sql.connect.dataframe.DataFrame = [MATNR: integer, MAKTX: string ... 5 more fields]
> vbap_clean: pyspark.sql.connect.dataframe.DataFrame = [VBELN: integer, POSNR: integer ... 3 more fields]

2) Drop Nulls:

b. dropna ()

MATNR should never be null.

✓ Yesterday (<1s) 23

```
mara_clean = mara_clean.dropna(
    subset=["MATNR"]
)

vbap_clean = vbap_clean.dropna(
    subset=["MATNR"]
)
```

> mara_clean: pyspark.sql.connect.dataframe.DataFrame = [MATNR: integer, MAKTX: string ... 5 more fields]
> vbap_clean: pyspark.sql.connect.dataframe.DataFrame = [VBELN: integer, POSNR: integer ... 3 more fields]

3) Fill Null values

c. fillna()

```

▶ ✓ Yesterday (<1s) 25

mara_clean = mara_clean.fillna(
    {
        "STATUS": "Unknown"
    }
)

#For sales:
vbap_clean = vbap_clean.fillna(
    {
        "NETWR": 0,
        "KWMENG": 0
    }
)

> mara_clean: pyspark.sql.connect.dataframe.DataFrame = [MATNR: integer, MAKTX: string ... 5 more fields]
> vbap_clean: pyspark.sql.connect.dataframe.DataFrame = [VBELN: integer, POSNR: integer ... 3 more fields]

```

4) Verify cleansed data:

d. Verify Cleaning

```

▶ ✓ Yesterday (2s)

print("MARA Before :", mara_df.count())
print("MARA After  :", mara_clean.count())

print("VBAP Before :", vbap_df.count())
print("VBAP After  :", vbap_clean.count())

> See performance \(4\)

MARA Before : 25
MARA After  : 24
VBAP Before : 50
VBAP After  : 49

```

7.3.6 Perform Transformation

Transform the data to meet the business reporting requirements.

The following transformations are performed:

- Select required columns
- Filter finished products (MTART = 'FERT')
- Create derived columns

1) Select Specific columns:

a. Select specific columns

```
Yesterday (<1s) 30

mara_select = mara_clean.select(
    "MATNR",
    "MAKTX",
    "ERSDA",
    "MTART",
    "MATKL",
    "STATUS"
)
```

mar_select: pyspark.sql.connect.dataframe.DataFrame = [MATNR: integer, MAKTX: string ... 4 more fields]

2) Filter

b. Filter

Only Finished Products

```
Yesterday (<1s) 32

mara_filter = mara_select.filter(
    col("MTART")=="FERT"
)
```

mar_filter: pyspark.sql.connect.dataframe.DataFrame = [MATNR: integer, MAKTX: string ... 4 more fields]

3) Derived column:

c. Derived Column

Material Age

```
Yesterday (<1s) 34

mara_transform = mara_filter.withColumn(
    "Material_Age_Days",
    datediff(current_date(), col("ERSDA"))
)
```

mar_transform: pyspark.sql.connect.dataframe.DataFrame = [MATNR: integer, MAKTX: string ... 5 more fields]

7.3.7 Aggregations

Aggregate sales data to generate business metrics such as total sales, total quantity sold, average selling price, and order count.

6. Aggregations

```

Yesterday (<1s) 38

vbap_group = (
    vbap_transform
    .groupBy("MATNR")
    .agg(
        sum("NETWR").alias("TOTAL_SALES"),
        sum("KWMENG").alias("TOTAL_QTY"),
        avg("Average_Price").alias("AVG_PRICE"),
        count("*").alias("TOTAL_ORDERS")
    )
)

> vbap_group: pyspark.sql.connect.dataframe.DataFrame = [MATNR: integer, TOTAL_SALES: long ... 3 more fields]

```

7.3.8 Join MARA and VBAP

Join the Material Master (MARA) and Sales Order Item (VBAP) datasets using the **Material Number (MATNR)** to create the consolidated Gold dataset.

7. Join MARA & VBAP

```

Yesterday (<1s) 40

gold_df = (
    mara_transform.alias("m")
    .join(
        vbap_group.alias("v"),
        "MATNR",
        "inner"
    )
)

> gold_df: pyspark.sql.connect.dataframe.DataFrame = [MATNR: integer, MAKTX: string ... 9 more fields]

```

7.3.9 Validation

Validate the transformed data by comparing the source and target record counts to ensure successful processing.

8. Validation

a. Silver vs Gold

```

▶ ✓ Yesterday (1s) 43

gold_count = gold_df.count()

print("Silver MARA :", mara_count)
print("Gold :", gold_count)

> See performance \(1\)

Silver MARA : 25
Gold : 15

```

7.3.10 Write Gold Layer

Write the final transformed dataset to the Gold layer of Azure Data Lake Storage Gen2 in CSV format.

9. Write CSV

```

▶ ✓ Yesterday (3s) 59

gold_df.coalesce(1) \
.write \
.mode("overwrite") \
.option("header", True) \
.csv(gold_path + "gold_MARA_VBAP.csv")

> See performance \(1\)

```

7.3.11 Create Delta Table

Create a Delta table in Unity Catalog from the transformed dataset to enable analytical querying.

10. Create Delta Table

```

▶ ✓ Yesterday (7s) 61

gold_df.write \
| .mode("overwrite") \
| .format("delta") \
| .saveAsTable("azure_databricks_ws.gold.final_output_MARA_VBAP")

> See performance \(1\)

```

7.3.12 Verification

Verify that the ETL process has completed successfully.

The following validation steps are performed:

- Verify the output files in the ADLS Gold layer.
- Query the Delta table to confirm successful creation and data availability.

1) Verify if data populated in ADLS Gold layer:

11. Verification

```

gold_verify_df = (
    spark.read
        .option("header", True)
        .csv(gold_path + "gold_MARA_VBAP.csv")
)

display(gold_verify_df)

```

> [See performance \(1\)](#) Optimize

gold_verify_df: pyspark.sql.connect.dataframe.DataFrame = [MATNR: string, MAKTX: string ... 9 more fields]

	MATNR	MAKTX	ERSDA	MTART	MATKL	STATUS	Material_Age_Days	TOTAL_SALES	TOTAL_QTY
1	100013	Router	2024-01-13	FERT	NETW	Active	911	160000	15
2	100010	Scanner	2024-01-10	FERT	OFFC	Active	914	60000	4
3	100002	Keyboard	2024-01-02	FERT	ELEC	Active	922	30000	15
4	100014	Switch	2024-01-14	FERT	NETW	Active	910	112000	14
5	100012	Speaker	2024-01-12	FERT	ELEC	Active	912	45000	17
6	100003	Mouse	2024-01-03	FERT	ELEC	Active	921	12600	18
7	100018	Tablet	2024-01-18	FERT	ELEC	Active	906	100000	5
8	100019	Docking Station	2024-01-19	FERT	ELEC	Active	905	18000	3
9	100016	UPS	2024-01-16	FERT	POWR	Active	908	32000	4
10	100004	Monitor	2024-01-04	FERT	ELEC	Active	920	150000	6
11	100021	Headphone	2024-01-21	FERT	ELEC	Active	903	40000	8
12	100001	Laptop	2024-01-01	FERT	ELEC	Active	923	405000	9
13	100017	Projector	2024-01-17	FERT	OFFC	Active	907	180000	4
14	100011	Webcam	2024-01-11	FERT	ELEC	Unknown	913	36000	9
15									

15 rows | 1.715s runtime Refreshed yesterday

2) Query Delta table to verify if the Delta table created successfully:

11. Verification

```

%sql
SELECT *
FROM azure_databricks_ws.gold.final_output_MARA_VBAP;

-- dataengineeringbatch17.dev.final_output;

```

> [See performance \(1\)](#) Optimize

	MATNR	MAKTX	ERSDA	MTART	MATKL	STATUS	Material_Age_Days	TOTAL_SALES	TOTAL_QTY
2	100010	Scanner	2024-01-10	FERT	OFFC	Active	914	60000	4
3	100002	Keyboard	2024-01-02	FERT	ELEC	Active	922	30000	15
4	100014	Switch	2024-01-14	FERT	NETW	Active	910	112000	14
5	100012	Speaker	2024-01-12	FERT	ELEC	Active	912	45000	17
6	100003	Mouse	2024-01-03	FERT	ELEC	Active	921	12600	18
7	100018	Tablet	2024-01-18	FERT	ELEC	Active	906	100000	5
8	100019	Docking Station	2024-01-19	FERT	ELEC	Active	905	18000	3
9	100016	UPS	2024-01-16	FERT	POWR	Active	908	32000	4
10	100004	Monitor	2024-01-04	FERT	ELEC	Active	920	150000	6
11	100021	Headphone	2024-01-21	FERT	ELEC	Active	903	40000	8
12	100001	Laptop	2024-01-01	FERT	ELEC	Active	923	405000	9
13	100017	Projector	2024-01-17	FERT	OFFC	Active	907	180000	4
14	100011	Webcam	2024-01-11	FERT	ELEC	Unknown	913	36000	9
15	100022	Microphone	2024-01-22	FERT	ELEC	Unknown	902	8000	3

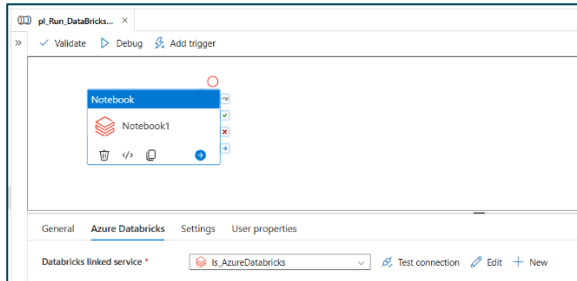
15 rows | 2.390s runtime Refreshed yesterday

This result is stored as `_sqlidf` and can be used in other Python and SQL cells. Send feedback

7.4 Build Azure Data Factory Pipeline

Build an Azure Data Factory (ADF) pipeline to orchestrate the execution of the Azure Databricks notebook, enabling automated data transformation and loading as per the business requirements.

Pipeline Design:



Pipeline Prerequisites:

7.4.1 Create Azure Databricks Access Token

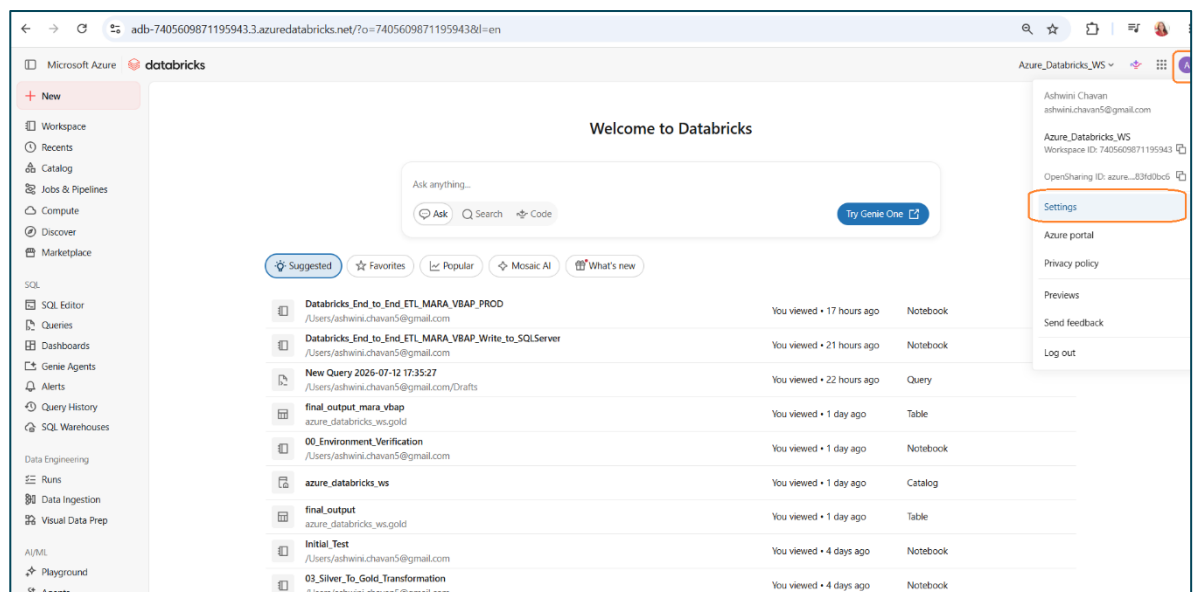
Purpose

Generate an access token to authenticate Azure Data Factory with Azure Databricks through the Databricks Linked Service.

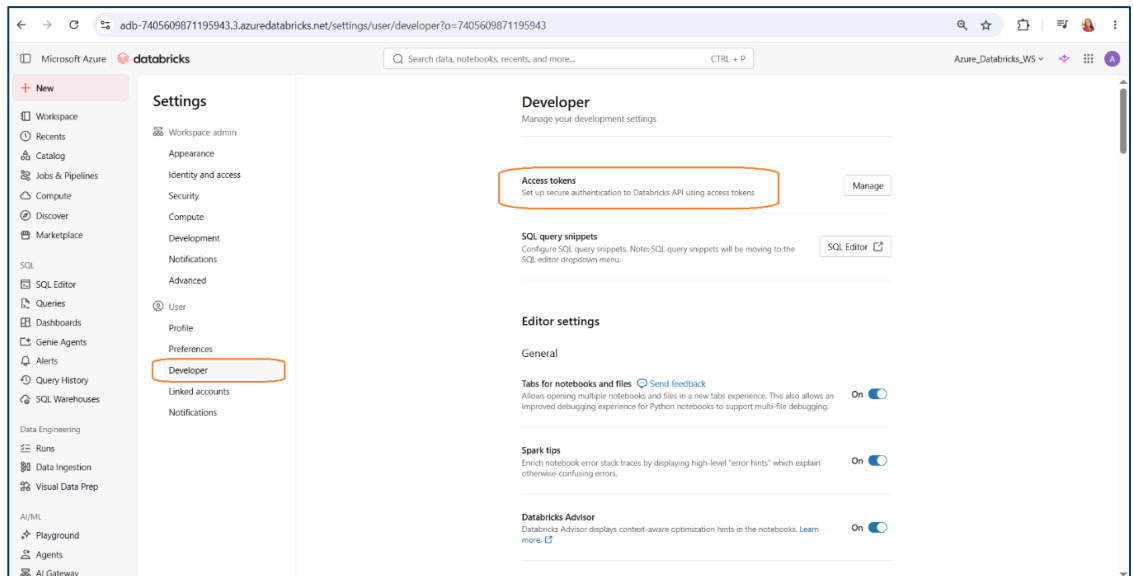
Steps

- 1) Open the Azure Databricks workspace

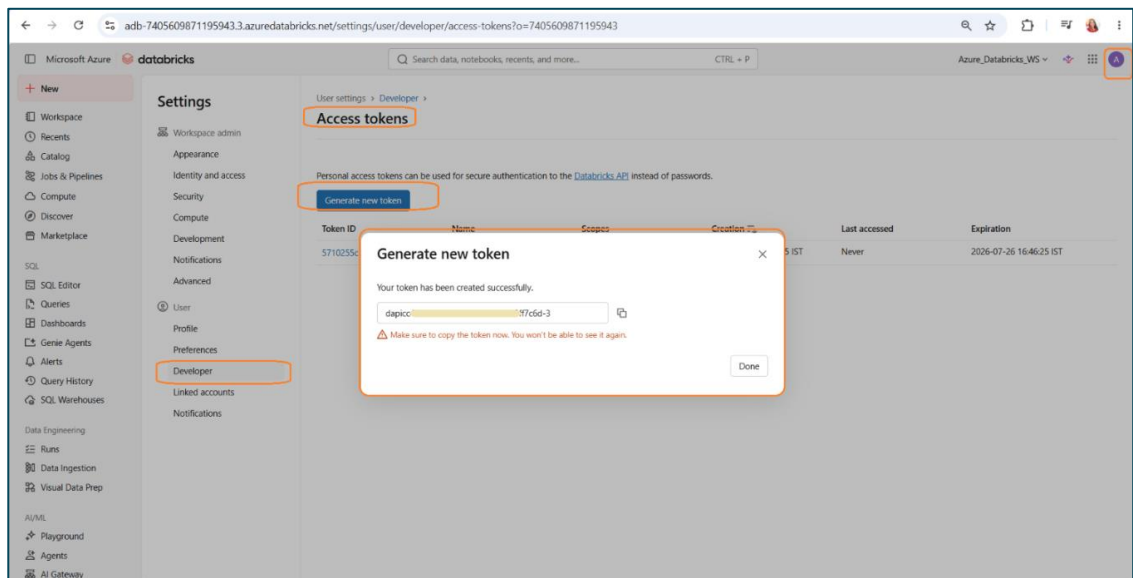
Navigate to **Settings** → **Developer** → **Access Tokens**.



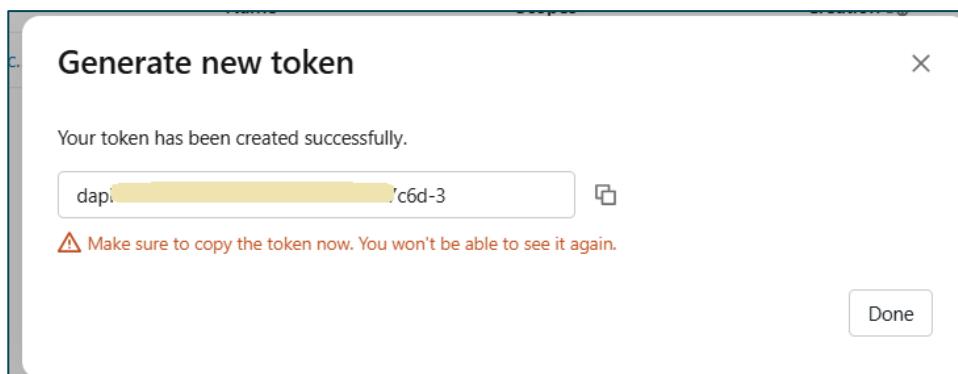
- 2) In settings, Navigate to Developers → Access tokens:



3) Click “Create Token”



4) Copy the generated access token



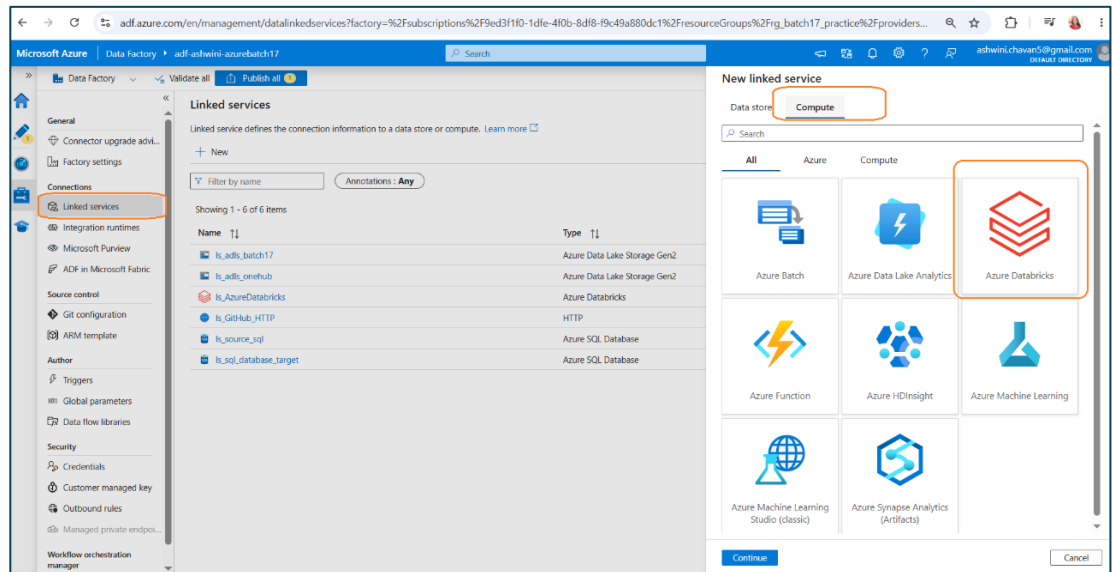
7.4.2 Create Azure Databricks Linked Service

Purpose

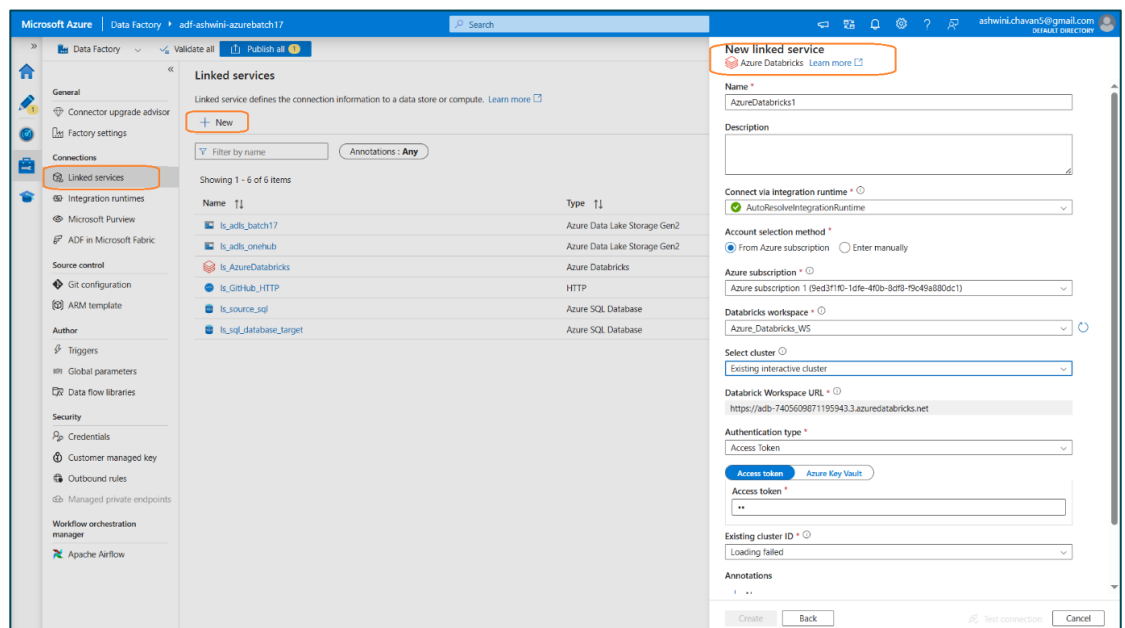
Create a Linked Service in Azure Data Factory to establish a secure connection with the Azure Databricks workspace.

Configuration

- 1) Open Azure Data Factory → Create new Linked service for databricks:
Click on “Continue”:



Following screen will be displayed:



2) Provide the following values:

Field	Value
Name	Any valid name
Description	Optional
Integration Runtime	AutoResolveIntegrationRuntime
Account Selection Method	From Azure Subscription
Azure Subscription	Azure Subscription
Databricks Workspace	Azure_Databricks_WS
Cluster Selection	Existing Interactive Cluster
Authentication Type	Access Token
Access Token	Generated in the previous step
Existing Cluster	ADF_Practice_Cluster

Click “Create”:

New linked service

Azure Databricks [Learn more](#)

Name *
AzureDatabricks1

Description

Connect via integration runtime * ⓘ
☒ AutoResolveIntegrationRuntime

Account selection method *
☒ From Azure subscription ☐ Enter manually

Azure subscription * ⓘ
 Azure subscription 1 (9ed3f1f0-1dfe-4f0b-8df8-f9c49a880dc1)

Databricks workspace * ⓘ
 Azure_Databricks_WS

Select cluster ⓘ
 Existing interactive cluster

Databrick Workspace URL * ⓘ
 https://adb-7405609871195943.3.azuredatabricks.net

Authentication type *
 Access Token

☒ Access token ☐ Azure Key Vault

Access token *

Choose from existing clusters * ⓘ
 ADF_Practice_Cluster

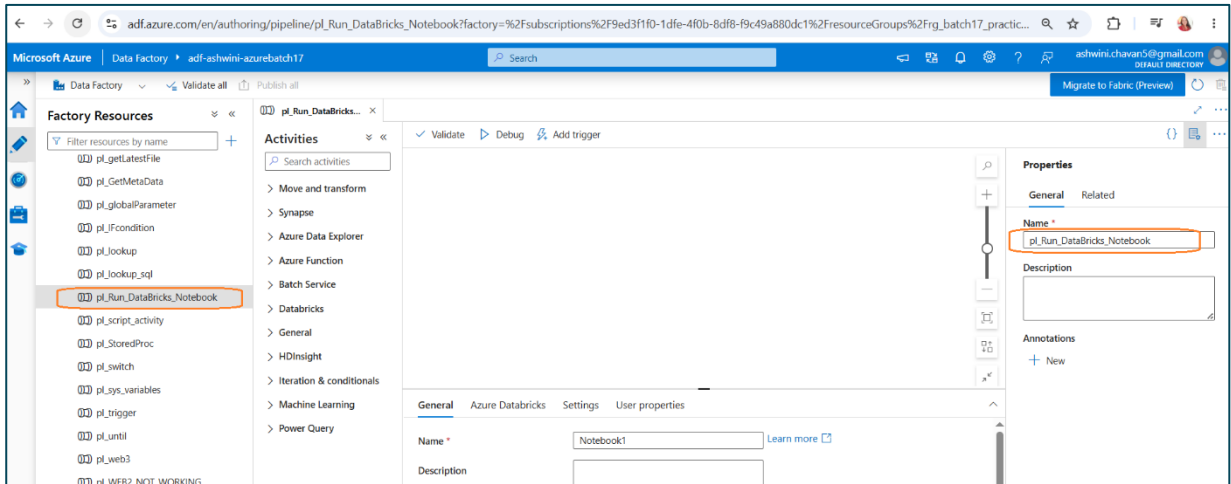
[Add dynamic content \[Alt+Shift+D\]](#)

Annotations

Pipeline steps:

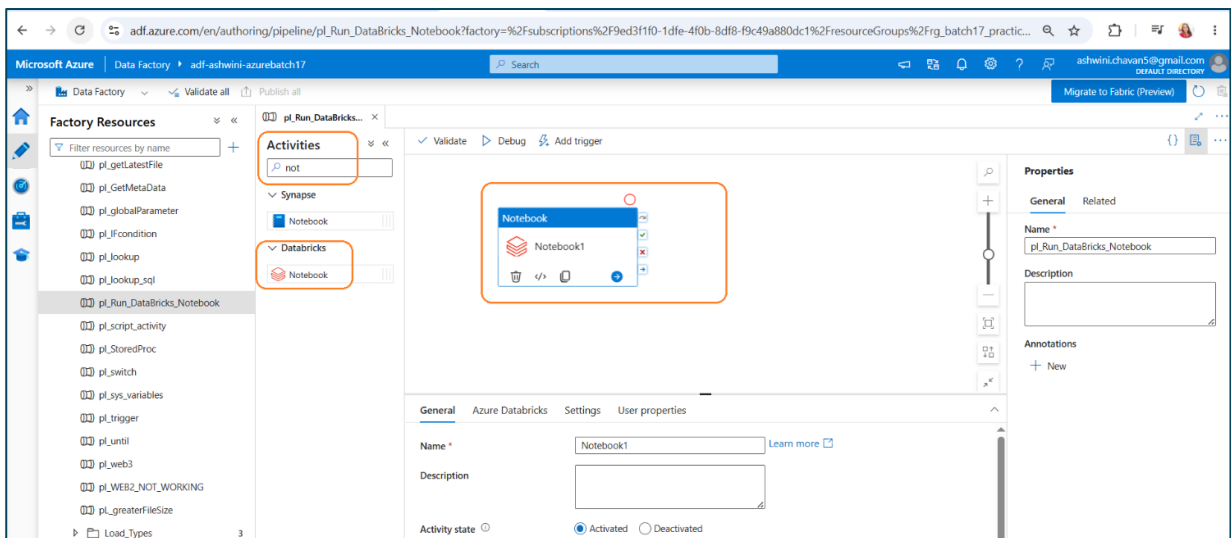
7.4.3 Create Pipeline

Create a new Azure Data Factory pipeline:



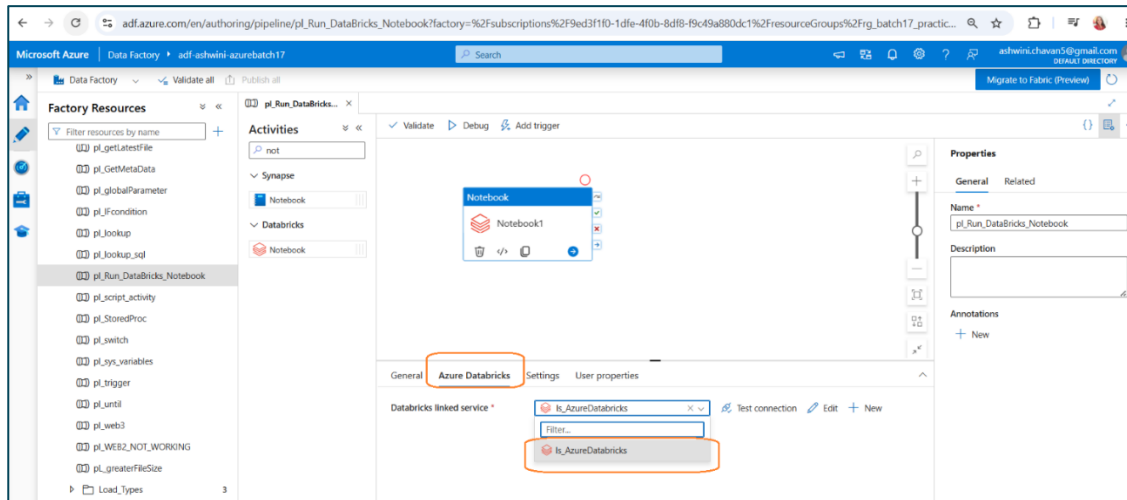
7.4.4 Add Databricks Notebook Activity

Drag the **Azure Databricks Notebook** activity onto the pipeline canvas.

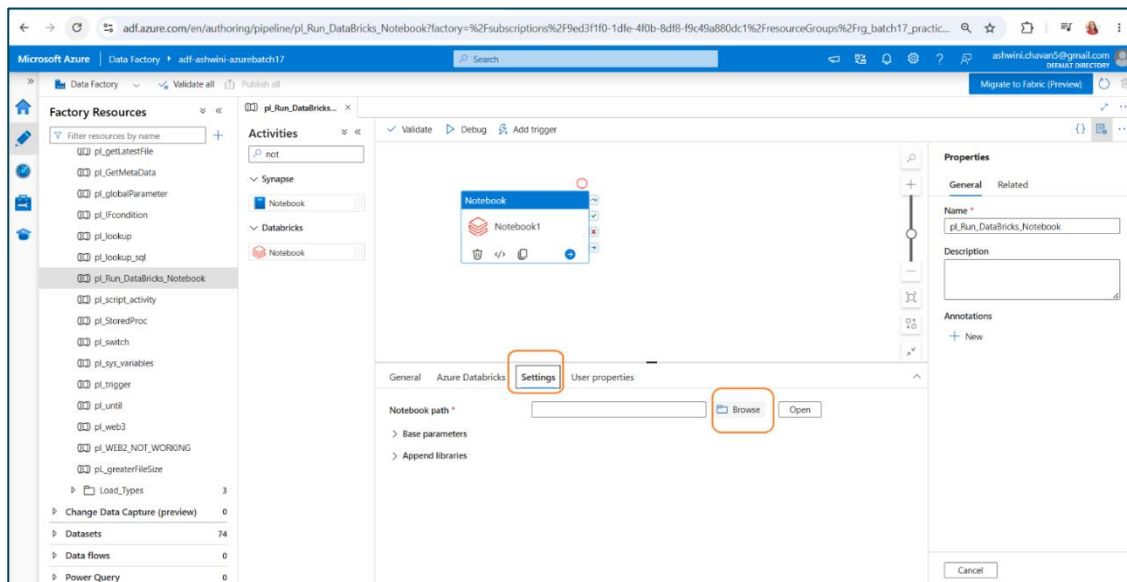


7.4.5 Configure the Notebook Activity

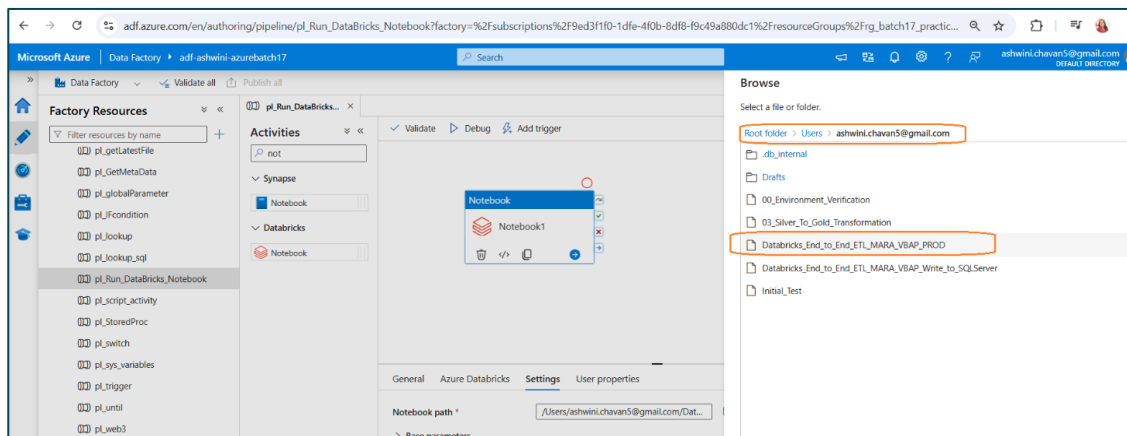
- 1) Select the previously created Azure Databricks Linked Service.



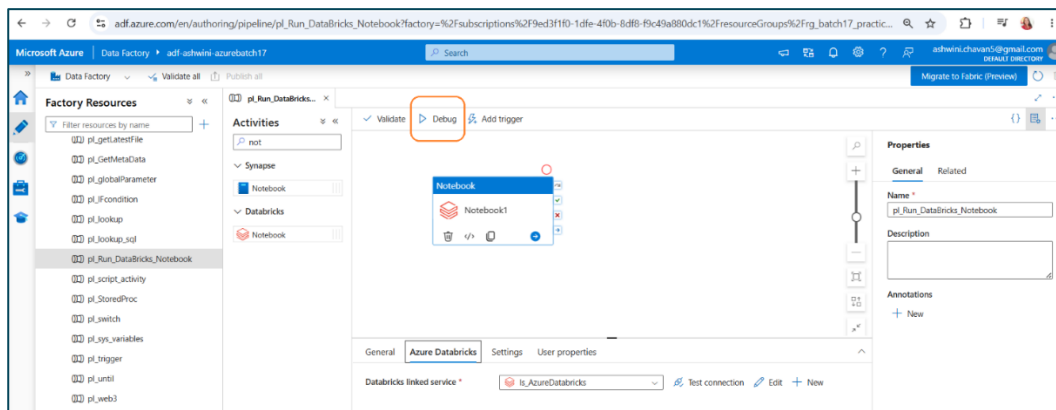
- 2) Navigate to the **Settings** tab and click **Browse**.



- 3) Browse the Databricks workspace and select the notebook to execute.



7.4.6 Execute Pipeline

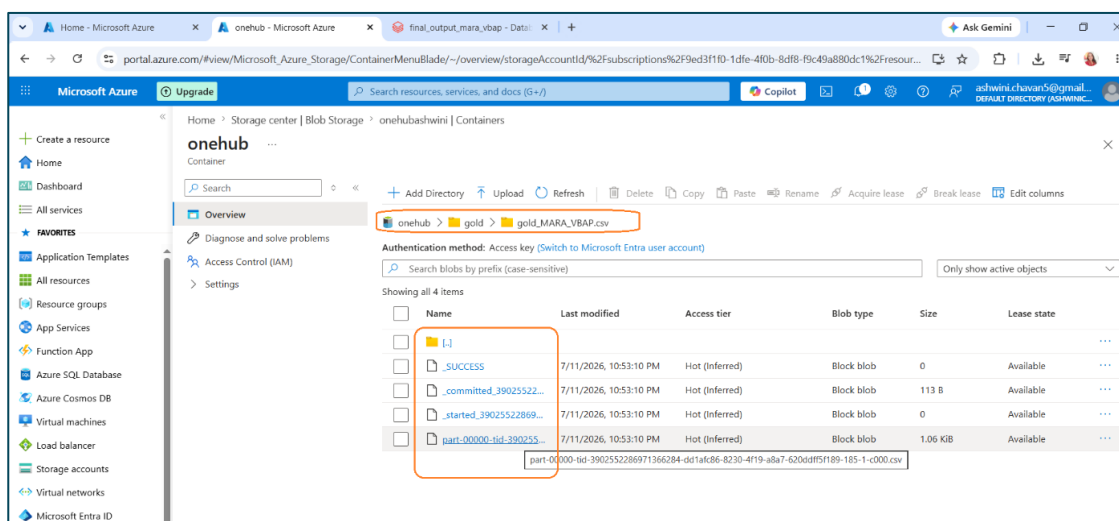
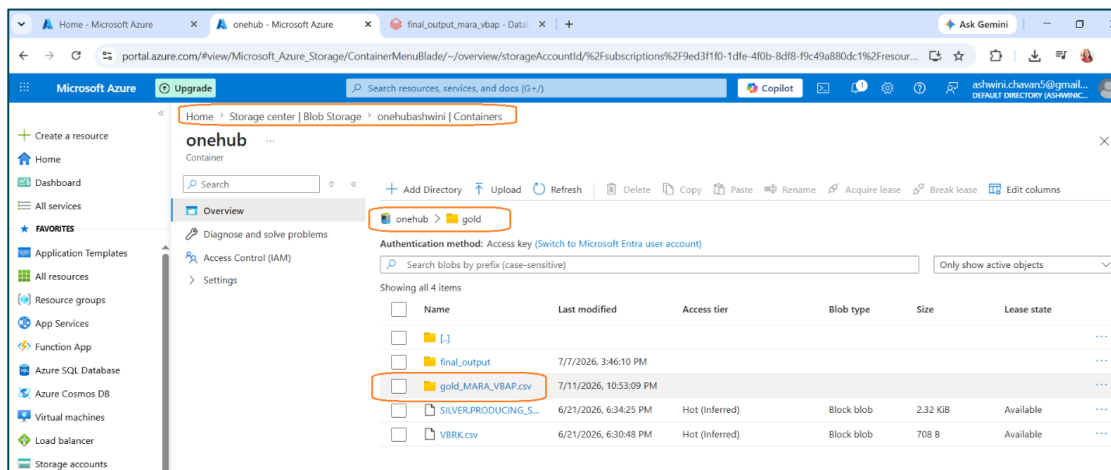


7.4.7 Verification

After successful execution, verify the following:

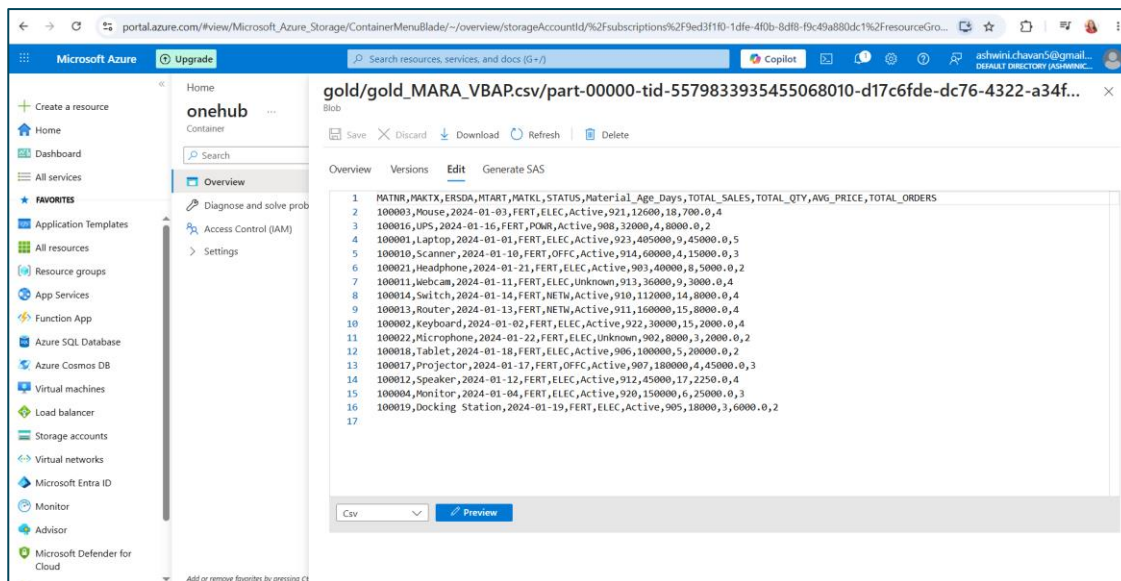
1. Gold Layer Output:

Verify that the output CSV file has been created successfully in the **Gold** layer of Azure Data Lake Storage Gen2.



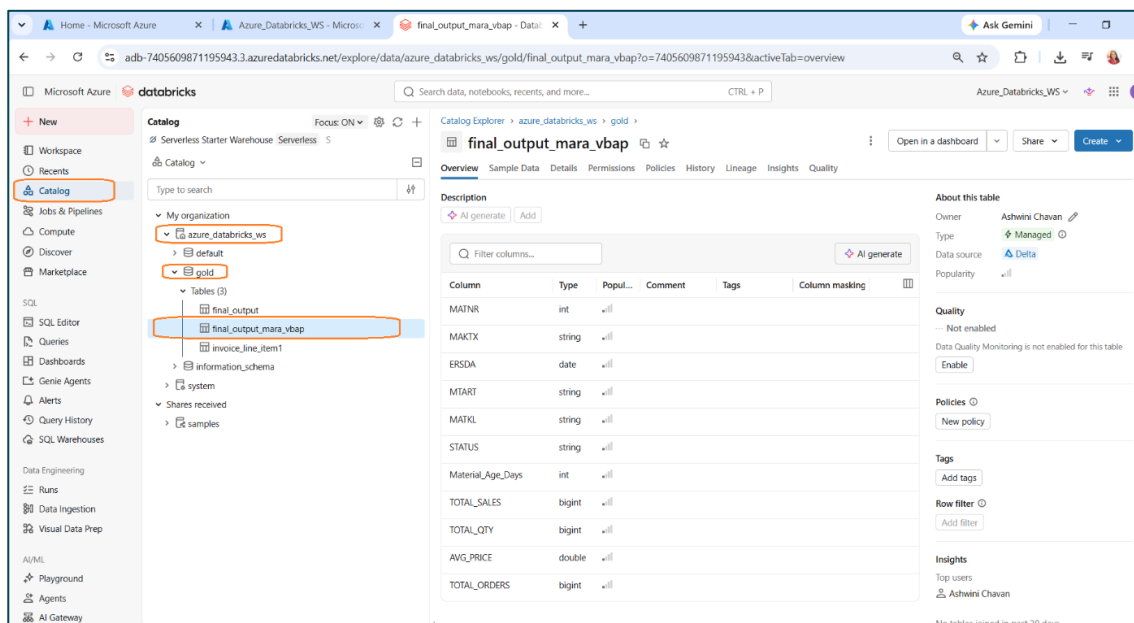
2. Validate Output Data

Open the generated CSV file and verify that the transformed data has been written successfully.



3. Verify Delta Table

Open Azure Databricks and query the Unity Catalog table to confirm that the Delta table has been created successfully.



8 Conclusion

This project demonstrated the implementation of an end-to-end ETL solution using Azure Databricks, Azure Data Factory, Azure Data Lake Storage Gen2, and Unity Catalog. The solution ingests data from the Silver layer, performs data profiling, quality checks, transformations, and validations using PySpark, and stores the processed data in the Gold layer as both CSV and Delta formats. Azure Data Factory orchestrates the notebook execution, resulting in a scalable, secure, and automated data processing pipeline suitable for analytical reporting and future enhancements.